ARQUIVO DE PRESERVAÇÃO • 30 OUTUBRO 2026

Motor Autônomo de Descoberta e Inteligência Técnica

Pesquisa em Git • Web • Fóruns • PDFs • Arquivos históricos • Engenharia de redes

Conteúdo: planejamento consolidado, decisões arquiteturais, comparações de ferramentas, precedentes, registro cronológico da conversa e reprodução integral dos quatro arquivos anexados.

> **Nota de fidelidade:** este PDF foi elaborado a partir de um arquivo mestre reconstituído com as informações acessíveis. Não é uma exportação literal de todas as mensagens. Preserve também o arquivo Markdown e, para histórico palavra por palavra, a exportação oficial do ChatGPT, se disponível.

Network Intelligence Engine — nome de trabalho
Git Eye + Web Eye + Autonomous Discovery Engine

Sumário de navegação

DOSSIÊ MESTRE — MOTOR AUTÔNOMO DE DESCOBERTA E INTELIGÊNCIA TÉCNICA

1. Resumo executivo e veredito final
2. Origem da ideia: o que o usuário pediu
3. Objetivos de produto e indicadores de sucesso
4. Arquitetura-alvo
5. Git Eye — mecanismo de pesquisa em código
6. Web Eye — mecanismo de descoberta web
7. Autonomous Discovery Engine — a “água encontrando caminhos”
8. Verificação cruzada — “três fontes” não são sempre três evidências
9. Processamento de conteúdo e biblioteca permanente
10. Ferramentas: decisão e comparações
11. Os precedentes: por que a ideia geral não é inédita
12. Exemplos de pesquisas técnicas que motivaram o projeto
13. Experiência de uso ideal
14. Riscos, limitações e regras de operação
15. Plano de desenvolvimento aprovado em princípio
16. Benchmark inicial de 20 perguntas técnicas
17. Decisões ainda abertas
18. Links e fontes mencionados na conversa
19. Relação com os relatórios e materiais anteriores
20. Estado final e recomendação de continuidade

REGISTRO CRONOLÓGICO DA CONVERSA

ANEXO B — QUATRO FONTES ORIGINAIS INTEGRAIS

DOSSIÊ MESTRE — MOTOR AUTÔNOMO DE DESCOBERTA E INTELIGÊNCIA TÉCNICA

Arquivo de preservação do projeto e da conversa · 30 de outubro de 2026
Nome de trabalho: Network Intelligence Engine
Componentes conceituais: Git Eye · Web Eye · Autonomous Discovery Engine · Biblioteca Permanente · Motor de Verificação Cruzada
Situação: especificação conceitual e pesquisa comparativa; **nenhuma implementação da plataforma foi afirmada como concluída**.

> **LEIA ANTES DE APAGAR O CHAT:** este material é uma **consolidação detalhada + registro cronológico reconstruído + cópias integrais dos quatro arquivos anexados**. Não é uma exportação nativa, byte a byte, da conversa. A íntegra exata de todas as mensagens, incluindo eventuais saídas de ferramentas e formatação original, só pode ser garantida por uma exportação oficial do histórico da conta, se disponível. Guarde o ZIP e, se a fidelidade literal de cada mensagem for indispensável, exporte os dados antes de excluir a conversa. Informações dos outros relatórios foram preservadas como **alegações das fontes**, não convertidas automaticamente em fatos verificados.

1. Resumo executivo e veredito final

O projeto propõe uma plataforma auto-hospedável para **pesquisar conteúdos técnicos difíceis de localizar** em repositórios Git, páginas web, fóruns, PDFs, manuais, sites históricos, bases de documentação e índices especializados, com possibilidade de incluir determinadas fontes `.onion` públicas e metadados públicos de BitTorrent quando houver finalidade legítima.

Em lugar de um único mecanismo que tente copiar a internet, o sistema combinará **buscas externas de largo alcance com um acervo próprio acumulativo, qualificado e pesquisável**. O usuário definirá **o que quer descobrir**; a plataforma deverá descobrir automaticamente **onde procurar**, seguir referências relevantes, registrar origens e verificar compatibilidade técnica.

Veredito acordado durante a conversa:

- A ambição é **tecnicamente plausível**, mas não existe evidência de uma aplicação open source única, pronta e completa que entregue toda a combinação com qualidade comprovada.
- **Não é necessário criar um buscador ou modelo de IA do zero.** Desenvolveremos a integração e o conhecimento específico sobre redes sobre ferramentas existentes.
- O traço mais original não é o crawler recursivo, pois essa abordagem já tem precedentes, mas a **especialização multivendor**, a **correlação técnica por equipamento/firmware** e a **memória contínua com evidências**.
- Não se deve anunciar que a plataforma pesquisa “literalmente a web inteira”, que é “a primeira do mundo” ou que uma informação foi comprovada só porque apareceu em três sites.
- O teste do produto deve medir **quantos problemas técnicos difíceis ele resolve com fontes corretas**, não a quantidade bruta de terabytes ou links armazenados.
- Deve começar com um **MVP enxuto**, escalar conforme medições e evitar redundância desnecessária entre softwares.

1.1. Frase central do projeto

“Uma investigação técnica autônoma que percorre trilhas de evidências, encontra materiais raros em diferentes fontes, verifica aplicabilidade por equipamento e versão, constrói um acervo permanente e explica tudo com referências verificáveis.”

1.2. O que o projeto não é

Não é um vírus, botnet, ferramenta de invasão, programa para contornar autenticação ou sistema que tenha acesso universal a dados privados. A semelhança com “vírus”, “raízes”, “água” ou “eletricidade” é **exclusivamente a expansão recursiva de caminhos a partir de referências públicas permitidas**.

---

2. Origem da ideia: o que o usuário pediu

O pedido inicial foi localizar **sistemas open source** disponíveis no GitHub, GitLab e outras fontes que conseguissem pesquisar **milhões ou bilhões de informações de repositórios** e outro conjunto de mecanismos que conseguisse pesquisar a **web amplamente**, incluindo Google, Bing, DuckDuckGo, fóruns, PDFs, documentos, sites, índices próprios, redes Tor e metadados ou conteúdos distribuídos por torrent.

O exemplo prático era um buscador excepcionalmente profundo para engenharia de redes e provedores de internet (ISPs):

- **Fabricantes:** Huawei, Datacom, Cisco, ZTE, VSOL, FiberHome, Intelbras, Digistar, TP-Link, Furukawa, MikroTik, SonicWall, Hillstone, entre outros.
- **Equipamentos:** switches, roteadores, OLTs, ONUs, firewalls, BNG/BRAS e concentradores PPPoE; especialmente Huawei NetEngine NE8000.
- **Tecnologias:** BGP, MPLS, PBR, RADIUS, PPPoE, CGNAT, GPON/XGS-PON, VLANs, redundância, roteamento, operação e interoperabilidade.
- **Casos difíceis:** redundância entre autenticadores BNG/BRAS, preservação de sessões, failover BGP, configurações raras de firmware e comandos CLI específicos de equipamento.
- **Formato desejado:** biblioteca crescente de gigabytes a milhares de gigabytes, organizada, pesquisável em linguagem natural e por comandos exatos, com download e preservação de fontes autorizadas.

O usuário acrescentou um princípio: encontrar a **mesma informação técnica em três fontes independentes** deve aumentar a confiança, sem ignorar que os três documentos podem ser cópias da mesma origem.

Depois acrescentou o requisito mais marcante: a plataforma **não deve depender de o usuário indicar sites**. Deve descobrir índices, pistas, nomes, citações, links e novas fontes, seguir cada caminho promissor e expandir as buscas em novas ramificações — **“como uma ramificação de uma árvore”, “uma eletricidade em uma madeira” e “como a água da chuva”.**

---

3. Objetivos de produto e indicadores de sucesso

3.1. Objetivos

1. Receber consultas simples ou investigações complexas em português e operar também com termos em inglês, espanhol e, se pertinente, chinês.
2. Pesquisar em **fontes federadas**, tanto existentes na internet quanto em arquivos previamente coletados.
3. Descobrir **novas fontes automaticamente**, inclusive por referências não clicáveis e nomes de protocolos/documentos.
4. Extrair texto de páginas, fóruns e documentos, preservar os originais quando permitido e manter histórico das versões.
5. Detectar e indexar **comandos CLI, exemplos de configuração, mensagens de erro, procedimentos, pré-requisitos e notas de compatibilidade**.
6. Relacionar cada descoberta ao **fabricante, produto, família, revisão, sistema operacional, firmware, release e contexto operacional**.
7. Corroborar afirmações por fontes independentes, detectar cópias, contradições e evidências insuficientes.
8. Produzir respostas e dossiês com **trechos citáveis, página, URL, data, origem, versão e estado de validação**.
9. Atualizar o acervo por **coleta incremental e campanhas agendadas**, preservando deduplicação e limites operacionais.
10. Ser **auto-hospedável, modular, auditável** e evolutivo.

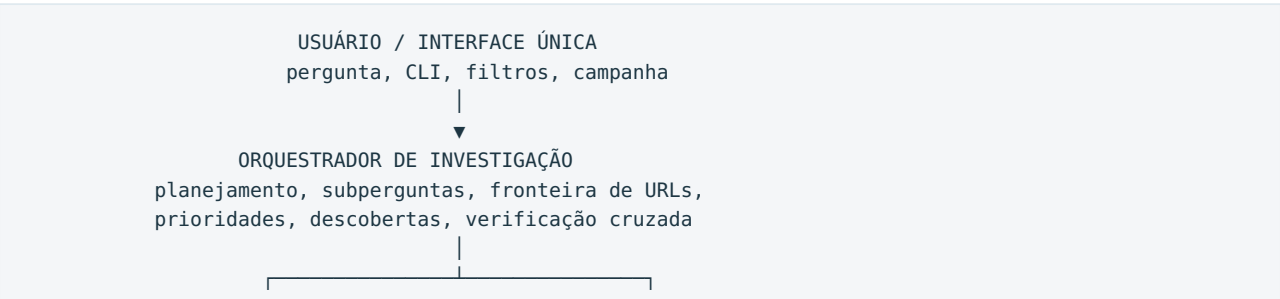
3.2. Critérios de sucesso

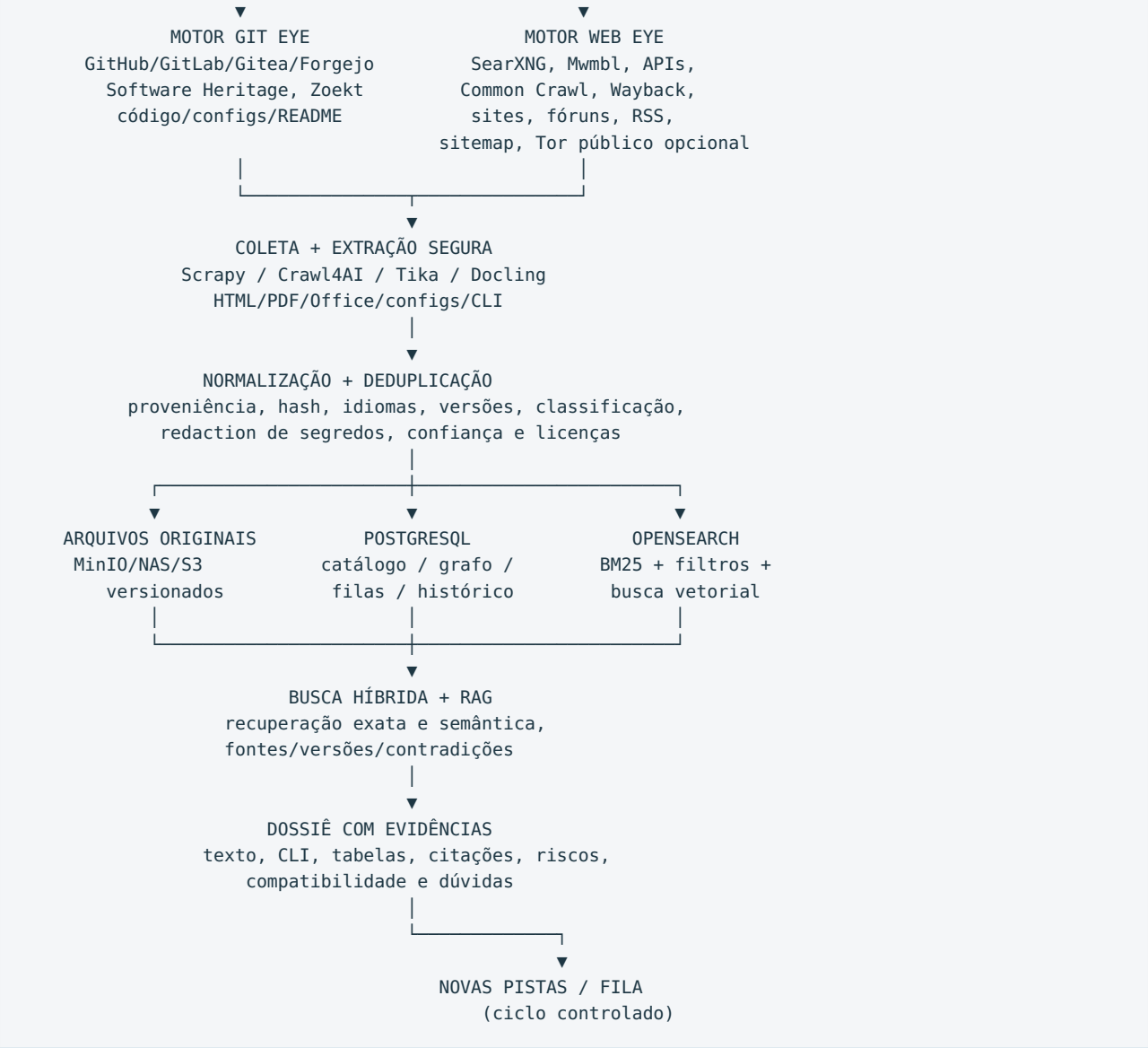
- Proporção de perguntas com **pelo menos uma fonte técnica relevante**, preferencialmente oficial.
- Precisão de **fabricante/modelo/firmware**, evitando misturar comandos incompatíveis.
- Correção da correspondência **comando → finalidade → modo operacional → pré-requisitos**.
- Reprodutibilidade da resposta: o leitor consegue localizar a seção original.
- Tempo e custo da pesquisa comparados com busca manual.
- Taxa de **fontes inéditas úteis** descobertas por expansão automática.
- Taxa de respostas sem evidência que o sistema corretamente **se recusa a afirmar**.
- Volume útil preservado vs. páginas repetidas/irrelevantes; custo de armazenamento.

Importante: latências, percentuais e tamanhos anunciados em relatórios externos são estimativas ou alegações que exigem ensaios reais; não são metas garantidas.

---

4. Arquitetura-alvo





4.1. Separação fundamental

- **Descoberta:** encontrar referências potenciais, URLs, nomes e repositórios.
- **Coleta:** obter legitimamente o conteúdo de fontes escolhidas.
- **Arquivamento:** preservar uma cópia do material, segundo licença e permissões.
- **Indexação:** criar estrutura de busca exata, estruturada e semântica.
- **Investigação:** avaliar e correlacionar documentos encontrados.
- **Resposta:** apresentar conclusão, condições de validade e citações.

Nenhuma ferramenta individual resolve necessariamente todas essas etapas.

---

5. Git Eye — mecanismo de pesquisa em código

Objetivo: localizar comandos, configurações, scripts, templates de automação, READMEs, exemplos de firmware e artefatos textuais em repositórios publicamente acessíveis ou autorizados.

Fontes de descoberta: APIs de busca GitHub/GitLab; projetos públicos em Gitea/Forgejo/Bitbucket e outras forjas; catálogos como ecosyste.ms; Software Heritage como arquivo histórico; eventualmente buscas de código públicas como grep.app ou Sourcegraph quando oferecidas e permitidas.

Índice próprio: Zoekt (motor especializado em código e texto, com substring/regex), alimentado por repositórios selecionados que a plataforma efetivamente baixou e indexou. OpenGrok é complementar para navegação por símbolos, referências e histórico; não é obrigatório no início.

Não confundir: Software Heritage preserva código-fonte em escala enorme, mas **não oferece automaticamente um grep instantâneo em todos os arquivos**. GH Archive contém principalmente **eventos públicos do GitHub** e não uma cópia integral pesquisável de todas as linhas de código. Zoekt precisa de conteúdo local ou pipeline de indexação preparado; não pesquisa literalmente todos os repositórios públicos sem aquisição/indexação.

Consultas exemplares:

```
"display bgp peer" "NE8000"
"radius-server" "pppoe" "huawei"
"/routing/bgp/connection" "RouterOS 7"
"ip policy route-map" "SonicWall"
"interface gpon" "OLT"
"DmOS" "BGP" "route-policy"
```

A sintaxe precisa de busca de cada motor será gerada conforme seu dialeto; os exemplos acima são **expressões de investigação**, não comandos de configuração validados.

---

6. Web Eye — mecanismo de descoberta web

Objetivo: alcançar documentação que uma busca comercial comum não prioriza: fóruns pequenos, arquivos históricos, posts antigos, PDFs e apresentações, listas de discussão, blogs técnicos, páginas em outros idiomas e portais de fabricantes.

Fontes e papéis:

- **SearXNG:** metabusca. Consulta múltiplos motores, mas não mantém uma cópia da web.
- **Mwmbl:** iniciativa open source de buscador com **índice próprio**. Complementa a descoberta, sem alegar cobertura comprovada de bilhões de páginas.
- **Common Crawl:** coleção pública massiva de capturas da web; índice de URLs e registros WARC/WAT/WET, não um mecanismo pronto de busca lexical instantânea em todo o conteúdo.
- **Wayback Machine / CDX:** localizar cópias de URLs ou sites antigos quando há capturas disponíveis.
- **Scrapy:** rastreamento direcionado, extração de links, sitemaps, filas e regras.
- **Crawl4AI:** extrair páginas e conteúdo preparado para IA, inclusive páginas modernas quando acessíveis.
- **Apache Nutch / Apache StormCrawler / Heritrix:** candidatos de expansão para crawling maior, distribuído ou preservação arquivística; **não instalar todos por padrão**.
- **YaCy:** alternativa P2P com crawler/índice próprio para experimentar, não componente obrigatório.
- **Ahmia:** pesquisas/crawling de serviços `.onion` públicos **conhecidos**, sem garantia de enumerar toda a rede Tor.
- **bitmagnet:** discovery e busca de **metadados BitTorrent/DHT**, não leitura universal do conteúdo de arquivos distribuídos.

Fóruns previstos, quando acessíveis e permitidos: Stack Overflow, Network Engineering e Server Fault (Stack Exchange), Viva o Linux, fórum MikroTik, Cisco Community, fóruns Huawei e de provedores, discussões públicas do GitHub, comunidades relevantes e conteúdos preservados no Wayback.

O sistema poderá **ler discussões, respostas, comentários, links e material referenciado**, dentro das condições de acesso e coleta de cada plataforma.

---

7. Autonomous Discovery Engine — a “água encontrando caminhos”

Este é o requisito que o usuário explicitou mais enfaticamente: a plataforma deve **descobrir novas fontes sem exigir que cada site seja indicado manualmente**.

7.1. Caminhos de descoberta

1. **Consultas amplas** a mecanismos externos e índices preexistentes.
2. **Links HTML** encontrados em artigos, páginas e respostas de fóruns.
3. **Referências em PDF, HTML e README** (URLs, autores, títulos, RFCs e nomes de documentação).
4. **Menções sem hyperlink:** termos e nomes de tecnologias/documentos geram novas buscas por título, modelo ou protocolo.
5. **Sitemap, RSS e Atom:** feeds e caminhos sinalizados pelos sites.
6. **Citações cruzadas:** vídeos, artigos, slides, repositórios e comentários que indicam mais material.
7. **Histórico:** URLs mortas levam à busca de capturas em arquivos históricos.
8. **Novas hipóteses:** a IA percebe lacunas e formula pesquisas derivadas.
9. **Atualizações:** revisitação de fontes importantes por alteração conhecida, feed ou agenda.

7.2. Fronteira de descoberta

Cada nova pista entra em uma fila contendo, idealmente:

`source_url`, `discovered_from`, `discovery_type`, `topic`, `vendor`, `model`, `firmware`, `language`, `candidate_score`, `depth`, `crawl_status`, `first_seen`, `last_seen`, `next_check`, `robots_policy`, `license`, `content_hash` e `parent_document_id`.

Prioridade: relevância para o tema, novidade, probabilidade de evidência útil, reputação e independência da fonte, compatibilidade de equipamento/versão e custo estimado de aquisição.

Segurança: limites por domínio e por campanha, taxa de requisições, profundidade, tamanho de resposta, timeout, filas de quarentena, detecção de traps de crawling, deduplicação por URL canônica e por hash, recusa de áreas privadas/não permitidas. Não usar rotação de identidade para contornar proibições.

7.3. O crescimento não deve ser “tudo indiscriminadamente”

A metáfora da árvore é a **expansão em grafo**. Uma página pode apontar para 20 links, que apontam para centenas de outros. Sem filtros, seriam milhões de páginas inúteis, repetidas ou armadilhas. O sistema deve priorizar **descoberta orientada ao assunto** (*focused crawling*), com orçamentos por tema/domínio, políticas de recursão e possibilidade de interromper ramificações pouco promissoras.

7.4. Exemplo de ramificação

1. Pergunta sobre redundância PPPoE no Huawei NE8000.

2. Busca encontra uma discussão sobre BNG HA.
3. Essa discussão cita determinado manual VRP.
4. O manual cita mecanismos de backup de assinantes e uma versão específica.
5. A plataforma procura esse mecanismo e encontra apresentações, scripts, outros fóruns e históricos.
6. Compara as versões e guarda documentos/citações relevantes.
7. Uma descoberta alimenta novas buscas, sem repetir o que já foi indexado.

Esse comportamento **não deve fazer downloads indiscriminados, acessar contas ou executar scripts**.

---

8. Verificação cruzada — “três fontes” não são sempre três evidências

O usuário propôs: **se a mesma informação aparece em três locais distintos, sua chance de ser verdadeira é maior**. Isso é um princípio útil desde que as fontes tragam **evidência independente e equivalente**, com escopo técnico comparável.

8.1. Classes de evidência sugeridas

1. **Não confirmado:** menção única ou conteúdo sem sustentação suficiente.
2. **Corroborado por fontes independentes:** resultados convergentes com proveniência diferente.
3. **Documentado oficialmente:** manual/fonte do fabricante indica suporte no modelo/release específico.
4. **Validado em laboratório:** execução reproduzida em ambiente autorizado e compatível.
5. **Conflitante ou não aplicável:** evidências divergentes, versão errada ou condição não satisfeita.

Essas classes **não substituem avaliação de qualidade**. Um fórum pode compartilhar uma configuração funcional, mas não provar aplicabilidade a outro firmware; um manual pode documentar um comando não habilitado na licença utilizada.

8.2. Independência de fontes

- Três páginas com mesmo texto copiado de um blog = **um grupo de origem**, não três confirmações independentes.
- Um manual oficial, um caso de suporte técnico independente e um teste de laboratório reproduzível = conjunto mais forte.
- Contar citações e **proveniência de origem**, não apenas URLs.
- Comparar **modelo, release de software, patch, feature, modo CLI, pré-requisitos e topologia**.
- Explicitar quando a evidência mostra somente “comando documentado” e não “solução completa comprovada”.

8.3. Exemplo crítico de redes

Redundância BGP, **failover de servidores RADIUS**, **alta disponibilidade do BNG/BRAS** e **continuidade/sincronização de sessões PPPoE** são assuntos relacionados, mas **não equivalentes**. Uma solução que mantém conectividade L3 não necessariamente preserva sessões de assinantes. O sistema precisa distinguir isso antes de propor comandos.

8.4. Saída da verificação

Uma afirmação deve trazer: resumo; evidências favoráveis/contrárias; URLs; seções ou páginas; origem comum detectada; fabricante; modelo; versão; risco de incompatibilidade; grau de validação; testes pendentes. Nenhuma ferramenta de IA deve dizer que um recurso funciona em produção sem a devida base.

---

9. Processamento de conteúdo e biblioteca permanente

9.1. Pipeline sugerido

`Descoberta → aquisição permitida → identificação de formato → extração → normalização → deduplicação → metadados → classificação técnica → busca lexical/vetorial → evidências citáveis → revisão incremental`.

- **Apache Tika:** texto e metadados em múltiplos formatos.
- **Docling:** estrutura de PDFs, tabelas e manuais (comparar com Tika na prática; podem operar como caminhos complementares).
- **ArchiveBox:** arquivamento de páginas selecionadas e seus artefatos quando permitido.
- **PostgreSQL:** catálogo e relacionamentos; trilha da coleta, versões e estado das evidências.
- **Object storage (MinIO/S3 compatível/NAS):** arquivos originais, extratos, capturas e versões.
- **OpenSearch:** busca textual BM25, filtros e possível busca híbrida/vetorial.
- **Zoekt:** índice especializado de texto/código em repositórios.
- **RAGFlow ou implementação própria:** camada opcional de RAG documental; decidir após protótipo.
- **GPT Researcher ou orquestrador próprio:** candidato para investigação; testar integração, custos e reprodutibilidade.

9.2. Preservação

Não basta salvar somente embeddings ou respostas produzidas por IA. Preservar **arquivo original (se permitido), hash SHA-256, URL, timestamp, parser utilizado, versão do extrator e número da página/seção**, além do texto estruturado. Documentos com mesma informação podem se relacionar sem duplicar o arquivo físico.

9.3. Estrutura física orientativa

```
/acervo/
  originais/
    github/
    gitlab/
    manuais-pdf/
    paginas-web/
    arquivos-historicos/
  extraidos/
    markdown/
    texto/
    tabelas/
    blocos-cli/
  capturas/
    warc/
    html/
  metadados/
    docs/
```

```
    releases/
    proveniencia/
  indices/
    opensearch/
    zoekt/
  campanhas/
    bgp/
    mpls/
    pbr/
    pppoe-bng/
    gpon/
```

Fabricante, modelo, versão e protocolo devem ser **campos de catálogo e facetas de pesquisa**, e não apenas pastas que obriguem a duplicar documentos.

9.4. Entidades do domínio

- `Source`: URL/forja/domínio/tipo de origem/política.
- `Document`: ID, hash, título, idioma, formato, licença, originais e revisões.
- `Citation`: documento, página/seção, trecho, afirmação suportada.
- `Vendor`, `ProductFamily`, `DeviceModel`, `OperatingSystem`, `FirmwareRelease`.
- `Command`: sintaxe canônica, contexto, tipo (consulta/configuração), argumentos, feature.
- `Feature`, `Protocol`, `Scenario`, `Topology`, `Prerequisite`.
- `Claim`, `Evidence`, `Contradiction`, `IndependenceCluster`, `ValidationRun`.
- `DiscoveryEdge`: de onde veio cada link/menção/documento.
- `ResearchCampaign`: pergunta, sementes, orçamento, critérios, resultados e revisões.

---

10. Ferramentas: decisão e comparações

As ferramentas listadas são **candidatas ou fontes**, não dependências todas aprovadas. Licenças, documentação e manutenção podem mudar; confirmar versões antes da instalação.

Projeto	Papel correto	Decisão do projeto
Zoekt	Pesquisa em código já indexado	**Núcleo Git**
SearXNG	Metabusca sobre motores externos	**Núcleo web**
Mwmbl	Índice independente da web	**Fonte integrada**
GitHub/GitLab APIs	Descoberta/busca/metadata de repositórios	**Fonte integrada**
Software Heritage	Arquivo e descoberta de código histórico	Fonte externa, não clone mundial
ecosyste.ms	Descoberta e metadados de repositórios	Complementar
Common Crawl	Arquivos públicos e índice de URLs	Fonte externa sob demanda
Wayback/CDX	Capturas históricas	Fonte externa sob demanda
Scrapy	Crawler programável	**Coleta principal**
Crawl4AI	Extrator web orientado a IA	**Coleta/extrator principal**
Tika	Parser de formatos variados	Principal conforme formato

Projeto	Papel correto	Decisão do projeto
Docling	Parsing estruturado de PDFs/tabelas	Principal conforme formato
ArchiveBox	Preservação de páginas	Complementar; não único storage
PostgreSQL	Catálogo, fluxo e proveniência	**Núcleo**
MinIO/NAS	Armazenamento de objetos/arquivos	**Núcleo**
OpenSearch	Pesquisa textual/híbrida	**Núcleo documental**
GPT Researcher	Investigação multietapas com IA	Candidato a coordenador
RAGFlow	Ingestão documental e RAG	Candidato a interface, comparar
Onyx	Pesquisa corporativa e conectores/RAG	Alternativa, não instalar junto por padrão
Apache Nutch	Crawl web massivo	Fase posterior
StormCrawler	Rastreamento contínuo distribuído	Fase posterior / experimento
Heritrix	Crawling de qualidade arquivística	Alternativa futura
YaCy	Crawler e índice P2P	Opcional, experimento
OpenGrok	Navegação avançada em código	Complemento ao Zoekt
Livegrep/Hound	Busca local regex em código	Alternativas ao Zoekt
Sourcebot	Interface/busca Git baseada em Zoekt	Cautela sobre mudanças de licença
Sourcegraph	Busca de código mais ampla	Não pressupor edição comunitária gratuita
GH Archive	Eventos públicos do GitHub	Descoberta, não texto integral
bitmagnet	Metadados BitTorrent/DHT	Fase opcional
Ahmia	Busca/crawling de onion público conhecido	Fase opcional
Paperless-ngx	Biblioteca documental e OCR	Alternativa para gestão PDF, não exigido
Diffbot	Crawler/knowledge graph comercial	Precedente tecnológico, não núcleo livre
Hyperresearch	Pesquisa com acervo permanente e ligações	Candidato a estudo, verificar implementação

10.1. Correções relevantes preservadas dos relatórios

1. **GH Archive ≠ código Git completo.** O BigQuery sobre eventos não possibilita grep direto em todos os arquivos de código.
2. **Software Heritage ≠ pesquisador universal de strings pronto.** Métricas de nós de grafo não equivalem a número de arquivos pesquisáveis; números de volume precisam da definição e data de cada dataset.
3. **Common Crawl ≠ Google full-text pronto.** Índices de URL/WARC não são a mesma coisa que pesquisa textual arbitrária em todos os registros.
4. **SearXNG ter muitos conectores ≠ todos funcionam em qualquer momento.** Rate limits, CAPTCHAs e bloqueios podem impedir consulta a certas fontes.
5. **Mwmbl “bilhões de páginas” não ficou comprovado** nos relatórios. Manter como índice complementar, com tamanho atual a verificar.

6. **Licenciamento importa:** Elasticsearch não deve ser simplificado como “distribuição padrão Apache 2.0”; Sourcebot e Sourcegraph têm avisos de mudança de distribuição/licença nos materiais. Validar versão e repositório antes da escolha.
7. **Instalação real:** exemplos antigos de Nutch com `inject` incompleto, `solrindex` deprecado/removido e `./OpenGrok index` não são receitas reproduzíveis. Não reutilizar sem checar documentação oficial.
8. **Zoekt: RAM/índice depende de corpus e workload.** Percentuais e fatores “3,5×” apresentados não devem ser tratados como garantias universais; planejar benchmark.
9. **Nutch não é requisito inicial.** Crawlers direcionados bastam para aprender quais fontes rendem evidência útil.
10. **Open Deep Research da LangChain arquivado:** os relatórios mencionaram arquivamento em agosto de 2026; verificar estado antes de basear projeto novo nele.
11. **Não inferir equivalência de tecnologias.** BGP, VRRP, RADIUS failover, Mesh e PPPoE session sync possuem semânticas e limites distintos conforme equipamento.
12. **Não confundir índice local com busca global.** Zoekt e OpenSearch trabalham no conteúdo que for efetivamente indexado neles.

Essas correções são **registro da discussão e dos relatórios**; detalhes de data e licença não foram revalidados como parte deste arquivamento.

---

11. Os precedentes: por que a ideia geral não é inédita

O usuário perguntou duas vezes se o projeto seria **“o primeiro do mundo”** e depois solicitou os precedentes concretos. A conclusão da conversa foi **não alegar ineditismo absoluto**.

Precedente	Função semelhante	Diferença / limite frente ao projeto
Diffbot	Crawler recursivo, índice web e grafo de conhecimento	Plataforma comercial; não nichada como motor aberto de engenharia multivendor
Hyperresearch (Jordan Gibbs)	Pesquisa, ligação entre documentos, biblioteca permanente e verificação de fontes	Necessária validação de maturidade, implementação e cobertura de coleta
GPT Researcher	Investigações e subperguntas automatizadas, relatórios com citações	Agente de pesquisa não é por si só crawler universal + acervo técnico especializado
YaCy	Busca/crawling descentralizado e índice próprio	Não é agente especializado que valida compatibilidade técnica
Apache StormCrawler	Crawling distribuído e contínuo, expansão por links	Infraestrutura de coleta, não a inteligência técnica fim a fim
Mwmbl	Buscador com índice próprio e coleta comunitária	Não é a plataforma de dossiês e validação por firmware proposta
Onyx	Pesquisa corporativa, conectores, RAG e agentes	Não é, isoladamente, um rastreador recursivo universal
Autonomous Research Agent (projeto experimental citado)	Declara investigação contínua e criação de hipóteses	Proposta experimental, maturidade não comprovada
RAGFlow	Ingestão e recuperação documental avançada	Não substitui descoberta global e controle de evidências multivendor
Juniper Marvis AI	Assistência e diagnóstico de redes no ecossistema Juniper	Especializado no próprio fabricante/ecossistema

Precedente	Função semelhante	Diferença / limite frente ao projeto
Cisco AI Canvas	Correlaciona evidências e ajuda investigações de rede	Produto comercial com escopo e integrações próprios

A inovação plausível: integrar descoberta recursiva em múltiplas fontes, persistência, deduplicação de origem, extração estruturada de CLI e **compreensão verificável de fabricante + modelo + versão + cenário de rede**. Ainda assim, pesquisas de anterioridade e concorrência seriam necessárias antes de declarar qualquer exclusividade ou pioneirismo.

---

12. Exemplos de pesquisas técnicas que motivaram o projeto

12.1. Huawei NE8000 como BNG/BRAS

Pergunta: “Como fazer redundância entre dois NE8000 para assinantes PPPoE, usando servidores RADIUS e failover de rotas BGP, considerando preservação de sessões?”

Investigar separadamente:

- Alta disponibilidade do encaminhamento de pacotes (routing / BGP).
- Autenticação e contabilização RADIUS, timeout, failover e estado das sessões.
- Mecanismos específicos de HA de BNG/BRAS documentados no modelo/release.
- Sincronização/preservação de sessões PPPoE, quando de fato suportada.
- Licenças, limitações de hardware, timers, precondições de topologia.

Não emitir uma receita única sem correspondência de modelo/VRP.

12.2. MikroTik RouterOS 7

Investigar PPPoE concentrator, BGP/VRF, RADIUS, políticas de roteamento, VRRP e a distinção entre continuidade de tráfego e continuidade de sessões PPPoE. Separar exemplos v6 e v7.

12.3. OLTs e redes GPON/XGS-PON

Huawei, ZTE, FiberHome, VSOL, Datacom, Intelbras, Digistar, TP-Link, Furukawa: identificar família, release, sintaxe de perfil, provisionamento e compatibilidade de ONUs. Não transferir comandos entre gerações distintas de OLT.

12.4. PBR e NGFW

SonicWall e Hillstone: pesquisar documentação de Policy-Based Routing e coexistência com BGP/ roteamento dinâmico, algoritmos de decisão, políticas e ordem de avaliação. Confrontar guia oficial e relatos operacionais.

12.5. Busca lexical exata

As expressões `display bgp peer`, `route-policy`, `interface gpon`, `/ppp secret`, MIB OIDs, nomes de parâmetros RADIUS e mensagens de erro são exemplos de conteúdo que exige busca **lexical** além de embeddings. RAG sem pesquisa exata pode trocar sintaxe e produzir resultados incorretos.

---

13. Experiência de uso ideal

O produto deve apresentar ao usuário uma **única caixa de pesquisa** com modos:

- **Busca rápida:** documentos, CLI, repositórios e fóruns com filtros.
- **Pesquisa profunda:** desmembrar um problema em subquestões, expandir links, comparar fontes e gerar relatório.
- **Monitoramento contínuo:** acompanhar fontes e assuntos que o usuário permitiu.
- **Biblioteca:** pesquisar somente o acervo local, com históricos e referências.
- **Comparador:** examinar diferenças entre versões de comandos, dois modelos ou duas hipóteses técnicas.
- **Auditoria de evidências:** ver quais URLs suportam cada afirmação e quais derivam da mesma origem.

Filtros úteis: fabricante, família de equipamentos, modelo, firmware, protocolo, idioma, documento oficial/comunitário, data, tipo de arquivo, confiabilidade, texto exato, regex e pesquisa semântica. Deve ficar visível a **origem** e o **estado de verificação**, sem vender “alta confiança” como certeza infalível.

13.1. Dossiê gerado

1. Pergunta e restrições interpretadas.
2. Resposta resumida com limitações.
3. Cenários/arquiteturas possíveis.
4. Comandos e trechos documentados, com versão e modo de execução.
5. Evidências pró/contra; cluster de fontes duplicadas.
6. Pré-requisitos e licenciamento relevante.
7. Plano de testes em laboratório, validação e rollback.
8. Fontes URLs, datas, PDFs com páginas/seções.
9. O que segue desconhecido.
10. Metadados e links do material preservado.

---

14. Riscos, limitações e regras de operação

14.1. Limitações incontornáveis

- Nenhum sistema conhece ou indexa **toda a internet**, cada repositório, cada fórum privado ou serviço onion não divulgado.
- Busca de documentos públicos pode esbarrar em autenticação, paywalls, remoções, `robots.txt`, APIs e termos.
- Fontes históricas não garantem arquivos preservados.
- Metadados de torrents não dão acesso automático ao conteúdo de arquivos e não legitimam downloads proibidos.
- A coleta deve respeitar direitos autorais e termos de uso. **Links e referências** podem ser preservados mesmo quando o arquivo integral não puder ser arquivado.
- Exemplo comunitário não garante que comando funcione em equipamento diferente.

14.2. Segurança do crawler

- Sem execução automática de scripts, firmwares, arquivos binários ou comandos CLI encontrados.

- Parsers e browsers em workers isolados, limites de CPU/RAM/disco/tempo e sem acesso à infraestrutura de produção.
- Inspeção de tipos de arquivo, limitação de downloads, checagem de origem e varredura de risco quando pertinente.
- Remoção/mascaramento de senhas, segredos, tokens, configs com dados pessoais e credenciais em material coletado.
- Sem violar autenticação, burlas técnicas, invasão, coleta de conteúdo privado ou redistribuição não autorizada.
- Política de retenção/remoção e trilha auditável de coleta.

14.3. Custo e escala

Uma biblioteca de **dezenas de GB a múltiplos TB** é uma ambição viável com dimensionamento adequado; um espelho integral do Git público ou da web é outra ordem de magnitude. Consumo de armazenamento depende de PDFs, repositórios, snapshots, extratos e índices; não afirmar fatores de compressão ou requisitos fixos sem medições. Preferir descoberta global **sem precisar copiar todos os dados**.

---

15. Plano de desenvolvimento aprovado em princípio

Fase 0 — Requisitos e prova de descoberta

- Definir 15 a 20 casos reais de redes difíceis de pesquisar manualmente.
- Montar inventário inicial pequeno de fontes oficiais e comunidades.
- Prototipar busca externa e índice local com fontes abertas permitidas.
- Registrar licenças e versões efetivamente testadas.

Aceite: localizar documentos relevantes e preservar URLs, páginas e versões corretamente.

Fase 1 — MVP funcional de pesquisa

- API e interface única.
- SearXNG (e API Git), Zoekt para repositórios selecionados, Scrapy/Crawl4AI, Tika/Docling.
- PostgreSQL + arquivos originais + OpenSearch BM25, filtros e histórico.
- Deduplicação por hash/URL; ingestão manual e programada; indicadores de qualidade.

Aceite: pesquisa exata e navegação entre documentos originais; nenhuma dependência de agente para simples pesquisa.

Fase 2 — Investigação multietapas e evidências

- Agente para gerar subperguntas e coordenar fontes.
- RAG híbrido com citações precisas.
- Extração/classificação de CLI por fabricante/modelo/versão.
- Comparação de evidências, clusters de cópia, contradições e estados de validação.
- Dossiê técnico exportável.

Aceite: respostas fundamentadas para casos difíceis sem misturar firmwares, com conflitos explicitados.

Fase 3 — Exploração autônoma ramificada

- Fila de URL/referência, fontes seed opcionais, links e menções, relevância por campanha.
- Revisitas incrementais, descoberta de novos índices e fóruns, sitemap/RSS, históricos.
- Controles de profundidade, taxa, domínios, custo, deduplicação e bloqueio de crawl traps.

Aceite: novas fontes úteis descobertas automaticamente, com proveniência e baixo desperdício.

Fase 4 — Escala

- Se houver demanda medida, testar Nutch/StormCrawler e workers distribuídos.
- Armazenamento de múltiplos TB, controle de retenção, backup, observabilidade.
- Fontes especializadas opcionais (Ahmia público, metadados BitTorrent, YaCy), sob justificativa concreta.

Aceite: throughput, custos e precisão monitorados; conteúdo irrelevante não domina os índices.

Fase 5 — Integrações futuras

- Possível integração com o ecossistema NexoISP e seus módulos de operação de rede.
- Permitir consulta de configurações **somente em ambientes autorizados**; execução em produção é um produto separado que exigiria autorização, revisões e controles adicionais.

Aceite: camada de conhecimento desacoplada e segura; não executar CLI automaticamente em produção.

---

16. Benchmark inicial de 20 perguntas técnicas

Não são testes já executados; são **casos recomendados** para avaliação futura.

1. Huawei NE8000: diferença entre redundância BGP e continuidade de sessões PPPoE.
2. Huawei NE8000: documentação BNG hot backup por modelo e versão VRP.
3. Huawei NE8000: autenticação RADIUS redundante vs. contabilização.
4. Huawei NE8000: route-policy e controle de rotas em cenários multihomed.
5. MikroTik RouterOS 7: diferenças de BGP em relação à versão 6.
6. MikroTik como PPPoE concentrator: riscos de failover e statefulness.
7. MikroTik RouterOS 7: PBR com routing rules vs. mangle.
8. Cisco: route-maps e policy-based routing dependente de plataforma.
9. Datacom DmOS: BGP policies documentadas por release.
10. Datacom: configuração SNMP/MIBs por família de equipamento.
11. OLT Huawei MA5800: perfis de linha/serviço por versão.
12. OLT ZTE C300/C320 vs. C600: diferenças reais de CLI.
13. FiberHome: provisionamento GPON por firmware.
14. V-SOL: perfis de VLAN e ONU por modelo.
15. Intelbras/Digistar/TP-Link/Furukawa: diferenças de CLI para OLT/ONU.
16. SonicWall: PBR e políticas de roteamento por firmware.
17. Hillstone StoneOS: PBR associado a BGP e precedência de regras.
18. MPLS L3VPN: configuração típica e compatibilidade de BGP/VRF.
19. Comunidades BGP: cenários de traffic engineering e diferenças por fabricante.

20. Pesquisa histórica: encontrar versão arquivada de manual técnico que mudou de URL.

Para cada pergunta, medir: cobertura de fontes úteis; número de **fontes independentes**; comandos específicos corretos; compatibilidade por release; links funcionais; contradições detectadas; rastreabilidade; custo/tempo; documentos de referência humanamente verificados.

---

17. Decisões ainda abertas

1. **Agente:** adaptar GPT Researcher, Hyperresearch, RAGFlow ou implementar orquestração própria? Precisa de comparativo e protótipo.
2. **Biblioteca RAG:** RAGFlow ou pipeline próprio sobre OpenSearch? Evitar duplicação e lock-in.
3. **Crawling distribuído:** StormCrawler, Nutch ou ampliação gradual de Scrapy? Medir primeiro.
4. **Modelos de IA:** local, cloud ou híbrido? Política de custos, privacidade e desempenho.
5. **Licenças exatas:** revisar por versão antes de incorporar componentes, inclusive redistribuição comercial futura.
6. **Orçamento de coleta:** limites por fonte, domínio, hora, campanha, tamanho e tipos de arquivo.
7. **Política de arquivo integral:** respeitar direitos autorais/licenças e exclusão de conteúdo restrito.
8. **Metadados estruturados:** taxonomia de fabricantes/modelos/firmwares, revisão humana quando ambíguo.
9. **Hardware:** definir após benchmark de índices, PDFs, OCR e análise de IA.
10. **Nome final e posicionamento:** “Network Intelligence Engine”, “Git Eye” e “Web Eye” são nomes provisórios, não marcas registradas aqui.

---

18. Links e fontes mencionados na conversa

Esses endereços são **pontos de partida históricos citados ao longo das mensagens e arquivos**. Eles não são certificação de que uma versão, política ou funcionalidade permanece inalterada.

18.1. Busca Git / código

- Zoekt — https://github.com/sourcegraph/zoekt
- OpenGrok — https://github.com/oracle/opengrok
- Sourcegraph — https://sourcegraph.com/
- Sourcebot — https://github.com/sourcebot-dev/sourcebot
- Livegrep — https://github.com/livegrep/livegrep
- Hound — https://github.com/hound-search/hound
- Software Heritage — https://www.softwareheritage.org/
- Software Heritage Archive — https://archive.softwareheritage.org/
- ecosyste.ms — https://ecosyste.ms/
- grep.app — https://grep.app/
- GitHub Code Search — https://github.com/search?type=code
- GitHub REST Search — https://docs.github.com/en/rest/search/search
- GitLab Search — https://docs.gitlab.com/
- GH Archive — https://www.gharchive.org/
- World of Code — https://worldofcode.org/

18.2. Web e coleta

- SearXNG — https://github.com/searxng/searxng
- Mwmbl — https://github.com/mwmbl/mwmbl
- Mwmbl Search — https://mwmbl.org/
- Common Crawl — https://commoncrawl.org/
- Wayback Machine — https://web.archive.org/
- Scrapy — https://github.com/scrapy/scrapy
- Crawl4AI — https://github.com/unclecode/crawl4ai
- Apache Nutch — https://github.com/apache/nutch
- Apache StormCrawler — https://github.com/apache/stormcrawler
- Heritrix — https://github.com/internetarchive/heritrix3
- YaCy — https://yacy.net/
- Crawlee — https://github.com/apify/crawlee
- Ahmia crawler — https://github.com/ahmia/ahmia-crawler
- bitmagnet — https://github.com/bitmagnet-io/bitmagnet

18.3. Extração, índices e conhecimento

- Apache Tika — https://github.com/apache/tika
- Docling — https://github.com/docling-project/docling
- ArchiveBox — https://github.com/ArchiveBox/ArchiveBox
- OpenSearch — https://github.com/opensearch-project/OpenSearch
- Apache Solr — https://solr.apache.org/
- PostgreSQL — https://www.postgresql.org/
- MinIO — https://github.com/minio/minio
- Qdrant — https://github.com/qdrant/qdrant
- Paperless-ngx — https://github.com/paperless-ngx/paperless-ngx
- RAGFlow — https://github.com/infiniflow/ragflow
- Onyx — https://github.com/onyx-dot-app/onyx
- Open WebUI — https://github.com/open-webui/open-webui
- AnythingLLM — https://github.com/Mintplex-Labs/anything-llm
- GPT Researcher — https://github.com/assafelovic/gpt-researcher
- Hyperresearch — https://github.com/jordan-gibbs/hyperresearch
- LangChain Open Deep Research — https://github.com/langchain-ai/open_deep_research

18.4. Precedentes e documentação de redes

- Diffbot — https://www.diffbot.com/technology
- Juniper Marvis — https://www.juniper.net/
- Cisco AI — https://www.cisco.com/
- Cisco Community — https://community.cisco.com/
- MikroTik Forum — https://forum.mikrotik.com/
- Viva o Linux — https://www.vivaolinux.com.br/
- Stack Overflow — https://stackoverflow.com/
- Network Engineering Stack Exchange — https://networkengineering.stackexchange.com/
- Server Fault — https://serverfault.com/
- RFC Editor — https://www.rfc-editor.org/

- RIPE NCC — https://www.ripe.net/
- NANOG — https://nanog.org/
- FRRouting — https://frrouting.org/

---

19. Relação com os relatórios e materiais anteriores

Quatro arquivos colocados no chat foram preservados **integralmente e sem alteração** no diretório `originais/` do ZIP:

- `Texto colado.txt` — consolidações de outras IAs, correções de Sourcebot/GH Archive, arquitetura e acréscimo posterior sobre Mwmbl.
- `Markdown colado (2).md` — múltiplos relatórios sobre Git, web, Tor, BitTorrent, Nutch, OpenGrok, documentos e síntese de ferramentas.
- `Texto colado (3).txt` — relatório extenso sobre rastreamento distribuído, Zoekt, Nutch, TorBot/TorCrawl e bitmagnet, incluindo afirmações técnicas que precisam validação.
- `Texto colado (4).txt` — pesquisa textual detalhada sobre Software Heritage, Code Eye/Web Eye, Common Crawl, Ahmia, bitmagnet, GPT Researcher e limitações.

Atenção a citações: alguns desses materiais contêm marcadores `⟦cite...⟧`, que só funcionavam em outras conversas ou respostas de IA. Foram mantidos como texto original, **não são referências bibliográficas automaticamente verificadas neste pacote**. Endereços e números específicos devem ser checados antes de usar em decisões de produção ou divulgação.

---

20. Estado final e recomendação de continuidade

Estado final em 30/10/2026: o usuário queria preservar todo o planejamento antes de excluir o chat. A direção conceitual foi aprovada para prototipagem, mas **não houve entrega de implementação funcional do motor nem medição de benchmark**.

Próxima ação recomendada: iniciar um repositório privado com este arquivo mestre e os originais, criar a especificação técnica do MVP e testar uma mesma bateria de perguntas complexas usando (a) busca convencional, (b) busca Git + web federada, e (c) investigação com fontes locais e verificação cruzada. Somente após isso decidir escala, arquitetura de agentes e funcionalidades adicionais.

Essência preservada: “Quero definir o assunto e deixar o sistema descobrir sozinho onde estão as evidências. Quero que ele siga ramificações de links, menções, PDFs e códigos, cruze fontes independentes, entenda comandos de redes por fabricante/firmware e construa uma biblioteca técnica permanente, confiável e crescente.”

REGISTRO CRONOLÓGICO DA CONVERSA

Tema: construção de um mecanismo autônomo de investigação técnica em Git, web, fóruns, PDFs e acervo local, voltado a redes e provedores.
Data de preservação: 30/10/2026.
Natureza do registro: reconstituição editorial abrangente, com trechos literais do usuário e síntese organizada das respostas. **Não é o histórico literal exportado pelo ChatGPT.** Os textos integrais dos quatro arquivos externos, que apareceram na conversa como anexos, estão preservados separadamente e reproduzidos no `ARQUIVO_MESTRE.md`.

Etapa 1 — Primeira pergunta: existe um “olho de Deus” para Git e toda a web?

Usuário (trechos literais e representativos):

> “Busca na web existe algum sistema opensource no git ou gitlab ou em outro lugar que você colocar ele para pequisar algo e ele pesquisar no git inteiro? milhoes ou bilhoes de informações?”
>
> “E existe outro ainda pior que procura em foruns, partes da web, pdf, drivers abertos, paginas, sites, em fim na web inteiro nao so no google qualquer coisa indexada na web, bing, google, duck duck, e até em redes tor e torrent?”
>
> “por exemplo usar esses dois sistema para pesquisa qualquer coisa que fale de comandos de redes, comandos de equipamentos de redes como varios tipos de sw huawei, datacom, cisco, olt como huawei, zte, vsol, fiberhome, intelibras, digistar, datacom, tplink, fukuwaka entre outros, tutoriais de redes como mpls, bgp, como fazer coisas especificas que quase nao tem no youtube e é dificil de encontrar como pbr, bgp, sonicwall, hilsstone, redudancia de bgp de autenticadores ne8000, redudancia micrkotik como autenticadores, etc... etc... realmente esses dois sistema ser um 'olgo de Deus' e montar suas bases de dados e pastas de gigas ou milhares de gigas de arquivos bem organizados etc...”

Resposta/encaminhamento neste ponto: foi solicitado um estudo comparativo aprofundado de projetos open source de busca Git, metabusca, crawling, arquivos históricos e coleta de documentos, com arquitetura para ISP/telecom. A solicitação de pesquisa abrangia Sourcegraph/Zoekt, OpenGrok, Software Heritage, SearXNG, YaCy, Nutch, Scrapy, Common Crawl, OpenSearch, Tika, ferramentas RAG e limitações de Tor/torrent.

Aprendizado: necessidade de distinguir metabusca, indexação própria de um corpus, pesquisa em índices públicos e arquivamento. Não se pode assumir acesso a todos os dados públicos ou privados da internet.

Etapa 2 — Quatro relatórios de outras IAs e descoberta do Mwmbl

Usuário (introdução representativa):

> “mais informações de outros chats, não sei se pode ajudar na pesquisa.”

O usuário apresentou **quatro materiais extensos**, com outras análises, URLs, comparações, sugestões e alertas. Os documentos integrais estão no ZIP:

1. `originais/Texto colado.txt`
2. `originais/Markdown colado (2).md`
3. `originais/Texto colado (3).txt`
4. `originais/Texto colado (4).txt`

O relato adicional sobre **Mwmbl** afirmava, em essência:

> “Ele trouxe uma descoberta que considero especialmente interessante para o nosso projeto: o Mwmbl, um mecanismo open-source que constrói seu próprio índice da web, em vez de simplesmente depender dos resultados do Google ou do Bing.”

E corrigia a afirmação de que Mwmbl já indexava bilhões de páginas: **não houve confirmação**. O texto mencionou que o projeto tinha anunciado *Seed Search*, integração com SearXNG e uma API. Também apresentava Nutch como opção para crawling de escala mais alta.

Síntese das correções encontradas nos materiais:

- **GH Archive:** eventos públicos, não espelho de código completo.
- **Sourcebot:** alerta de mudança de licença a partir de versões novas, exigindo verificação.
- **Open Deep Research (LangChain):** relato de arquivamento do repositório; não seria base preferencial sem verificar.
- **Software Heritage:** não confundir grafos de objetos com número de arquivos únicos pesquisáveis.
- **Common Crawl:** os índices não são um buscador full-text pronto para toda a web.
- **Elasticsearch:** licenciamento não se resume à distribuição padrão Apache 2.0.
- **SearXNG:** integrações suportadas não são motores simultaneamente operacionais garantidos.
- **Nutch e OpenGrok:** exemplos de instalação encontrados estavam incompletos/desatualizados.
- **Zoekt:** requisitos de RAM e tamanho de índice exigem benchmark, sem percentuais universais.

Encaminhamento da conversa: o assistente informou que as novas informações seriam acrescentadas a uma pesquisa consolidada. A conversa posterior passou a pedir que **não fosse mais utilizada a modalidade de Pesquisa Aprofundada**.

Etapa 3 — “Não use mais a pesquisa profunda”; escopo, opinião e veredito

Usuário (literal):

> “não use mais a pesquisa profunda, você concorda com isso? qual o escopo, opnião, veredito final?”

Resposta — decisões principais:

- Aprovação da **direção geral**, não de um amontoado de ferramentas ou de integrações sem teste.

- Encerramento da seleção exploratória ampla como fase suficiente para iniciar especificação e benchmark.
- Arquitetura com **Git Eye + Web Eye + agente coordenador + acervo permanente + mecanismo de validação técnica**.
- **Zoekt**, **SearXNG**, **OpenSearch**, **PostgreSQL** e armazenamento de arquivos apontados como componentes centrais candidatos; **Crawl4AI/Scrapy**, **Tika/Docling** para coleta e extração.
- **Mwmbl, Software Heritage, Common Crawl e Wayback** como fontes externas; **Nutch/StormCrawler** como expansão; **Ahmia/bitmagnet** como fontes opcionais.
- **GPT Researcher e RAGFlow** como candidatos, não como dupla obrigatória.
- Diferenciar BGP, RADIUS, BNG/BRAS e sessões PPPoE nas pesquisas, evitando falsas equivalências.
- Começar com protótipo e aproximadamente **20 problemas reais de telecomunicações**, medindo qualidade, origem, compatibilidade e custos.

Veredito registrado: o objetivo não é copiar a web; é achar uma informação rara, correta e verificável em um acervo potencialmente enorme.

Etapa 4 — “Teremos que desenvolver?”

Usuário (literal):

> “então não há uma solução real para o que queremos, teremos que desenvolver ela? e se sim, do que ela será capaz?”

Resposta — decisão principal:

O projeto poderá ser construído como **plataforma própria de orquestração**, sem reimplementar os índices ou os crawlers fundamentais. Foi sugerido o nome de trabalho **Network Intelligence Engine**.

Capacidades planejadas:

- Pesquisa federada em Git/web/PDFs/arquivos próprios.
- Investigação multietapas em linguagem natural.
- Descoberta de fontes e acompanhamento de atualizações.
- Preservação, deduplicação e versionamento do acervo.
- Extração e classificação de comandos CLI, exemplos de configuração e documentação.
- Comparação por fabricante/modelo/firmware e verificação cruzada.
- Dossiês técnicos com fontes, limitações, riscos, testes e rollback.

Separação em entregas: MVP de pesquisa unificada; versão de investigação inteligente; versão de expansão contínua/autônoma.

Limites: não garante acesso à internet inteira, não descobre conteúdo privado, não prova automaticamente que comandos funcionam em produção e não dispensa testes específicos.

Etapa 5 — Verificação cruzada e fóruns públicos

Usuário (literal):

> “para saber se uma coisa é verdade ou não basta comparar com outros achados se eu encontrar a mesma coisa em 3 locais distintos a chance de ser verdadeiro é maior principalmente trantando de algo tecnico.”
>
> “será capaz de entrar em foruns se tiverem abertos como stack oveflow, viva linux etc...? será de fato uma busca de Deus! ?”

Resposta — decisões principais:

Sim, pretende-se pesquisar **fóruns abertos** quando permitido: Stack Overflow, Stack Exchange (Network Engineering, Server Fault), Viva o Linux, fórum MikroTik, Cisco Community e outras fontes técnicas. O software deverá **abrir tópicos, ler respostas e referências, seguir links e guardar evidências**, sujeito a API, acesso público e regras do site.

Três URLs com o mesmo texto copiado podem representar apenas **uma evidência original**. O motor deverá rastrear dependência das fontes. Proposta de estados:

- Não confirmado.
- Corroborado por fontes independentes.
- Documentado oficialmente para modelo/versão.
- Validado em laboratório.
- Conflitante/não aplicável, quando houver evidências contrárias.

Também nasceu a ideia de **pesquisa investigativa recursiva**: novas descobertas revelam lacunas, que viram novas consultas; o acervo retém o histórico.

Formulação que ficou estabelecida: não basta mostrar links; deve **investigar, comparar, preservar e explicar**.

Etapa 6 — Rastreamento como árvore, água ou eletricidade

Usuário (literal):

> “mas eu terei que ficar indicando onde ele irá pesquisar ou ele irá sair encontrando indices, rastro, sites, menções e links, se alguem colocar um link ele vai atras e vai sair rastreando tudo como um virus?”
>
> “como uma ramificação de uma arvore uma eletricidade em uma madeira encontrando caminhos como uma agua da chuva?”

Resposta — principal requisito novo:

Não será necessário indicar manualmente cada site: um conjunto inicial de temas e fontes pode se expandir por descoberta recursiva. A plataforma deverá rastrear links HTML, links citados em PDFs e README, sitemaps, feeds, citações e **menções não clicáveis** que geram novas consultas. Isso se aproxima de *focused crawling*, *graph exploration* e *recursive discovery*.

O assistente explicou que o crescimento pode ser combinatório: por exemplo, 20 novos links distintos em cada nível dão 20, 400, 8.000, 160.000 e 3.200.000 novos caminhos em cinco níveis, se **não houver repetição nem filtragem**. Esse é um exemplo matemático hipotético, não uma previsão do volume real.

Decisão de segurança: expansão controlada por relevância, limites, crawl traps, URL canônica, deduplicação e políticas de sites. **Não** replicar o programa, invadir hosts, contornar autenticação ou executar arquivos recebidos.

Nova peça conceitual: Autonomous Discovery Engine, com fila inteligente e política de revisita, capaz de crescer independentemente da presença do usuário, se o servidor permanecer ativo e autorizado a realizar essas tarefas.

Etapa 7 — “Seria o primeiro do mundo?”

Usuário (literal):

> “seria o primeiro do mundo? @Busca na web”

Resposta — veredito sobre originalidade:

Não. Existem precedentes robustos de crawlers web e agentes de investigação. A resposta destacou **Diffbot** (crawler/index e knowledge graph), **GPT Researcher**, **Hyperresearch**, **RAGFlow** e um **Autonomous Research Agent** experimental como soluções que já cobrem partes relevantes da proposta.

A diferenciação potencial está em pesquisar documentação técnica **multivendor**, extrair compatibilidade por firmware/CLI, preservar conhecimento e cruzar evidências independentes.

Também foram citados concorrentes adjacentes de redes:

- **Juniper Marvis AI Assistant:** assistente para diagnóstico no ecossistema Juniper.
- **Cisco AI Canvas:** investigação, correlação de evidências e automação no ecossistema Cisco.

Conclusão: há oportunidade, mas não se deve alegar ser o primeiro a inventar crawling recursivo ou pesquisa profunda com IA.

Etapa 8 — Pedido explícito dos precedentes

Usuário (literal):

> “@Busca na web 'Originalidade da ideia geral\nJá existem precedentes' quais ?”

Resposta — sete precedentes enumerados:

1. **Diffbot:** crawling recursivo, descoberta de páginas e grafo de conhecimento; solução comercial.
2. **Hyperresearch:** investigação, memória documental e referência cruzada; conferir maturidade e implementação.
3. **GPT Researcher:** investigação/subperguntas com IA e relatórios.
4. **YaCy:** buscador/crawler com índice próprio, incluindo modelo P2P.
5. **Apache StormCrawler:** infraestrutura de rastreamento contínuo e distribuído.
6. **Mwmbl:** índice da web e descoberta comunitária.
7. **Onyx:** RAG, conectores e investigação empresarial.

A comparação destacava que **cada peça já existe em alguma forma**, mas não foi demonstrada uma solução geral de classificação/validação de CLI especificamente por modelo/firmware multivendor integrada a todas elas.

Mudança da recomendação: além de GPT Researcher e RAGFlow, testar **Hyperresearch + StormCrawler + Zoekt + OpenSearch** como alternativa, evitando reescrever funções maduras; o papel do nosso código será a orquestração e a inteligência técnica específica.

Etapa 9 — Solicitação de preservação antes da exclusão

Usuário (literal):

> “preciso salvar completamente esse chat, pesquisa, planejamentos, comparações, precedentes
>
> salvar essa conversa inteira, recomenda por .md .pdf ou outro arquivo? preciso que você gere um arquivo porque irei apagar essa conversa!”

Resposta documental nesta etapa: recomendar Markdown como fonte principal de conhecimento portátil, PDF como leitura/consulta, ZIP como pacote de preservação contendo fontes originais integrais, dossiê, registro cronológico e índice de referências. Explicar com transparência o limite da reconstrução textual vs. exportação literal do ChatGPT. **Nenhum arquivo do pacote deve ser interpretado como prova de uma instalação já existente.**

Síntese das decisões em uma linha

Git Eye + Web Eye → descoberta recursiva por links/menções → extração e preservação → PostgreSQL + arquivos + índices especializados → verificação por fontes independentes/ modelo/firmware → relatório técnico citável → novos caminhos de pesquisa.

Os 10 princípios que não podem se perder

1. Não prometer literalmente “toda a web”.
2. Não exigir que o usuário indique cada fonte manualmente.
3. Encontrar informações raras em fóruns, Git, PDFs e material histórico.
4. Fazer **expansão recursiva com limites e relevância**.
5. Guardar originais permitidos, índices, proveniência e histórico.
6. Tratar diferentes modelos/releases como material técnico diferente.
7. Cruzar fontes e **não contar cópias como evidências independentes**.
8. Diferenciar dados documentados, relatos comunitários e testes reproduzidos.
9. Usar componentes open source existentes e construir a especialização/integração.
10. Validar por benchmark real antes de afirmar escala, qualidade ou exclusividade.

ANEXO B — QUATRO FONTES ORIGINAIS INTEGRAIS

Os quatro blocos a seguir reproduzem **literalmente o conteúdo de bytes decodificados em UTF-8** dos arquivos que o usuário forneceu. Os arquivos separados, com os bytes originais, estão preservados no ZIP em `originais/`. O conteúdo dos blocos abaixo é material de terceiros/outras IAs e pode conter informações incorretas ou citações sem destinos verificáveis.

Anexo B.1 — Texto colado.txt

Origem: arquivo enviado pelo usuário; reprodução literal do conteúdo textual (sem edição).

```
Recebi os relatórios das outras IAs, incluindo os dois arquivos .txt. Eles acrescentaram ferramentas importantes que não tínhamos considerado inicialmente, principalmente Sourcebot, Apache Nutch, Apache Tika, Heritrix, OpenGrok, bitmagnet e Paperless-ngx.

A direção está ficando mais clara: precisamos de dois motores especializados — Git e Web — conectados a uma biblioteca permanente e a um agente de pesquisa profunda.

Antes de consolidar tudo, encontrei três correções importantes nas pesquisas:
1. GH Archive não é um espelho completo do código do GitHub. Ele armazena eventos públicos, como commits, issues e forks. Serve muito bem para descobrir atividade e repositórios, mas não permite pesquisar diretamente todas as linhas de código pelo BigQuery.
   gharchive.org+1
2. Sourcebot deixou de ser totalmente open-source nas versões novas. Em 8 de outubro de 2026, o projeto anunciou que o desenvolvimento a partir da versão 6 seria fechado. As versões anteriores continuam disponíveis. Portanto, eu manteria o Zoekt como núcleo aberto de longo prazo.
   Sourcebot+1
3. O Open Deep Research da LangChain foi arquivado em 21 de agosto de 2026. Ainda serve como referência de arquitetura, mas não seria minha primeira escolha para iniciar um projeto novo.
   GitHub
Isso mostra por que não devemos simplesmente somar todas as recomendações das IAs. Algumas ferramentas são redundantes, outras mudaram de licença e algumas afirmações confundem descoberta de dados com indexação integral.
Consolidação técnica: o que realmente vale a pena aproveitar
Projeto Decisão Justificativa
SearXNG Essencial       Metabusca em diferentes fontes
Zoekt   Essencial       Índice próprio para pesquisa exata de código e configurações
Software Heritage       Essencial como fonte externa    Descoberta e recuperação de código histórico
Common Crawl    Essencial como fonte externa    Acervo histórico de páginas da internet
Crawl4AIEssencial       Extração de conteúdo de sites e fóruns
ArchiveBox      Essencial       Preservação das páginas encontradas
Apache Tika     Essencial       Identificação e extração de conteúdo de múltiplos formatos
Docling Essencial       Extração estruturada de manuais, tabelas e PDFs
GPT Researcher  Essencial       Agente de pesquisa multietapas
RAGFlow Alta prioridade Biblioteca de conhecimento com IA
OpenSearch      Alta prioridade Índice textual e semântico de documentos
Apache Nutch    Segunda fase    Rastreamento em massa e expansão distribuída
OpenGrokComplementar    Histórico e navegação entre símbolos do código
Paperless-ngx   OpcionalExcelente gerenciador documental, mas não obrigatório junto ao RAGFlow
bitmagnet       Terceira fase   Descoberta de metadados na rede BitTorrent
Ahmia Crawler   Terceira fase   Pesquisa em serviços Tor públicos conhecidos

O RAGFlow merece atenção especial. Suas atualizações de 2026 acrescentaram recursos de pesquisa em múltiplas etapas, compilação de conhecimento e ingestão de conteúdo web. Isso o torna um candidato particularmente forte para o acervo documental.
GitHub
```

Já o Apache Nutch continua bastante relevante para coleta em massa, inclusive com plugins para Apache Tika e mecanismos de indexação. Mas começar por ele adicionaria complexidade desnecessária antes de sabermos quais sites e fabricantes apresentam melhores resultados.
Apache Nutch™

Outras correções importantes
Software Heritage: o snapshot de junho de 2026 realmente registra aproximadamente 58,7 bilhões de nós e 1,1 trilhão de relações. Entretanto, os nós representam diferentes objetos do grafo de software; não são 58,7 bilhões de arquivos independentes. O grafo compactado ocupa aproximadamente 16 TiB.
Software Heritage documentation

Zoekt e RAM: um relatório recomenda memória equivalente a 20% do corpus, mas a FAQ do Zoekt recomenda 20% a mais que o tamanho do corpus para o servidor de busca. Isso não é um limite absoluto, porque o motor usa arquivos mapeados em memória, porém significa que grandes índices podem sofrer bastante com falta de cache.
GitHub+1

Common Crawl: ele não oferece uma pesquisa textual universal instantânea em bilhões de páginas. Seu índice público é principalmente orientado a URLs e registros WARC. Uma pesquisa por uma expressão rara exige localizar conjuntos candidatos e processar conteúdo, o que pode custar tempo e recursos.
Common Crawl

SearXNG: centenas de integrações disponíveis não significam centenas de mecanismos operacionais simultâneos. Alguns podem retornar CAPTCHA, erros ou bloqueios, e a saída JSON precisa estar habilitada.
SearXNG Documentation (2026.10.25+54ca1d9cf)+1

Além disso, eu retiraria GitHound e SpiderFoot do núcleo principal. São mais orientados a investigações de segurança e OSINT do que à construção de uma enciclopédia de comandos e protocolos. Também não trataria exemplos de VRRP, MESH e sincronização PPPoE como soluções equivalentes de redundância sem comprovação específica por plataforma.
A arquitetura provisória mais equilibrada
Agente pesquisador
GPT Researcher + ferramentas próprias

Motor Git
GitHub/GitLab, Software Heritage, ecosyste.ms, Zoekt

Motor Web
SearXNG, Common Crawl, Crawl4AI, Wayback

Processamento e preservação
Apache Tika, Docling, ArchiveBox, classificação, deduplicação e validação

Biblioteca permanente
Armazenamento de objetos + PostgreSQL + OpenSearch + RAGFlow

Conhecimento técnico verificável
Comandos CLI, fabricantes, firmware, protocolos, tutoriais, configurações, evidências e referências

Eu acrescentaria uma capacidade que nenhum dos relatórios explorou suficientemente: a validação técnica automatizada das descobertas.

O sistema não deveria apenas dizer que encontrou um comando Huawei ou Datacom. Ele precisaria distinguir se o comando é de visualização ou configuração, a família do equipamento, a versão do sistema operacional, os pré-requisitos e se existe comprovação em documentação oficial. Uma IA pode ajudar a classificar essas informações, mas não deve inventar compatibilidade nem executar comandos diretamente em equipamentos de produção.

Essa camada seria particularmente valiosa para utilizar futuramente o acervo nas integrações com OLTs,

BRAS e demais equipamentos do NexoISP.
Minha avaliação até agora
Os outros relatórios melhoraram bastante a cobertura da pesquisa. No entanto, ainda não considero a arquitetura definitiva.

Faltam três decisões de engenharia importantes: comparar o RAGFlow com uma implementação própria sobre OpenSearch, estabelecer uma estratégia de descoberta contínua sem depender excessivamente de buscadores comerciais e definir como validar comandos de diferentes versões de firmware.

Também precisamos testar a qualidade real das pesquisas, não apenas o número de repositórios ou páginas anunciados pelos projetos.

O objetivo final não deve ser simplesmente armazenar 10 TB de arquivos. Deve ser encontrar uma informação técnica rara, correta e verificável em meio a esses 10 TB.

Os relatórios que você trouxe já oferecem uma boa base para isso.

mais informações de outros chats, não sei se pode ajudar na pesquisa.# Busca em Repositórios Git (código-fonte)

Para procurar em repositórios Git/GitLab de forma massiva, existem motores de busca de código open source. Um exemplo é o **OpenGrok**, um mecanismo de busca e cross-reference de código-fonte escrito em Java. Ele permite indexar múltiplos repositórios (SVN, Git, Mercurial etc.) locais e fazer buscas por termos no código. Projetos como o OpenGrok podem ser instalados em servidor próprio: basta clonar ou baixar os repositórios (por exemplo, via git clone para cada projeto) e configurar o OpenGrok para indexá-los. O OpenGrok é “rápido e utilizável” e suporta navegar por históricos de versão. Outras ferramentas similares incluem motores open source como **Sourcegraph** (versão comunitária gratuita para repositórios públicos) ou mesmo soluções simples de grep/ripgrep sobre cópias locais, mas para bases de “milhões de arquivos” é recomendado usar um índice estruturado. Em resumo, o OpenGrok e ferramentas equivalentes podem indexar gigabytes de código e disponibilizar interface web de busca dentro de sua própria base de dados de código.

Indexação e Busca na Web Profunda

Para indexar a web (sites, fóruns, PDFs, etc.) de forma independente de Google/Bing, recomenda-se um crawler + índice próprios. Por exemplo, o **Apache Nutch** é um rastreador web open source altamente extensível e escalável. O Nutch pode “beber” de seeds (lista de URLs iniciais), navegar links recursivamente e extrair texto (com Apache Tika, por exemplo) de páginas HTML, PDF, docs, etc. Ele pode ser integrado a um índice como o **Elasticsearch** ou **Apache Solr**. O Elasticsearch é um mecanismo de busca distribuído open source, “projetado para velocidade e escala”; assim, ao combinar Nutch (para coletar dados) e Elasticsearch (para indexar e pesquisar), é possível montar um sistema de busca de web de grande porte. O Nutch é usado em projetos de escala de web: por exemplo, o Common Crawl adotou o Nutch para rastrear centenas de milhões de páginas, o que comprova sua capacidade de lidar com bilhões de documentos.

Além disso, existem projetos de busca alternativa/comunitária. Por exemplo, o **Mwmbl** é um motor de busca independente, não lucrativo e open source. Ele mantém um índice próprio (atualmente com bilhões de páginas, segundo seus dados) e é construído por voluntários. O Mwmbl permite pesquisar o conteúdo da web por palavras-chave usando sua própria infraestrutura. Ele reforça que é “open source” e “sem fins lucrativos”, com o índice e ranking gerenciados pela comunidade. Além disso, há ferramentas descentralizadas: por exemplo, o **YaCy** é um buscador P2P open source em que cada nó compartilha parte do índice (ou seja, indexação distribuída entre usuários). Também existem metabuscais como

SearXNG (agrega resultados de Google, Bing etc. de forma privada) — mas esses não mantêm índice próprio.

Para buscas na **rede Tor** (serviços .onion), pode-se usar o Ahmia, um buscador open source para o Tor. O Ahmia é um mecanismo de busca específico para domínios ocultos .onion e é mantido como software livre. Seu código (disponível no GitHub) inclui crawlers que coletam conteúdo dos sites na rede Tor e indexam-no com Elasticsearch. Assim, para pesquisar fóruns e páginas dentro do Tor, o Ahmia (ou outros rastreadores .onion personalizados) são as opções conhecidas. Já para **torrents**, não há um motor de busca universal open source conhecido; seria necessário implementar um rastreador de trackers ou DHTs e indexá-lo (uma tarefa complexa). Em geral, buscadores de torrent comerciais existem (p. ex. The Pirate Bay), mas a tarefa de montar um índice torrent open source envolveria baixar arquivos .torrent de fontes públicas e extrair texto (se o conteúdo o permitir) ou metadados.

Ferramentas e Recursos de Indexação

- **Apache Nutch + Elasticsearch/Solr**: crawling genérico da web, com capacidade de indexar PDFs, fóruns, blogs, etc. É escalável a bilhões de páginas (via MapReduce ou clusters). Exemplos de uso: o projeto Common Crawl usa Nutch para indexar centenas de milhões de sites. Nutch pode ser configurado com parsers (PDF, Word via Tika) para indexar documentos inteiros em um índice de texto completo.
- **Sphinx ou OpenSearch**: motores open source de busca textual que podem indexar documentos extraídos por crawlers. Por exemplo, Elasticsearch (licença Apache) é popular e “distribuído, open source”, suportando bilhões de documentos. O Sphinx (sphinxsearch.com) também é livre e otimizado para textos, incluindo indexação JSON.
- **YaCy**: rede P2P de busca onde cada usuário contribui ao índice global. Não há servidor central; cada nó mantém parte do índice. É bom para anonimato e resiliência, mas o índice total tende a ser menor.
- **Mwmbl**: projeto comunitário (index próprio). É como um “Google descentralizado” mantenido por voluntários. Útil para busca geral na web sem rastreamento de dados pessoais.
- **SearXNG**: metabusca open source, que consulta simultaneamente vários buscadores (Google, Bing etc.) e agrega resultados. Não indexa diretamente, mas não rastreia o usuário. É fácil de instalar (Docker), mas depende das fontes externas.

Instruções para Montar Bases Locais

Para criar seu próprio sistema de busca local, siga estes passos:

1. **Coleta de dados**: Descubra fontes de informação (Git repos, sites, fóruns, PDFs). Por exemplo, clone repositórios Git para diretórios locais (um por pasta) se for buscá-los. Para sites e fóruns, configure um crawler (como Apache Nutch ou Scrapy) que comece em URLs iniciais (seeds) e salve o conteúdo em arquivos de texto ou JSON. Para PDFs e manuais, baixe os arquivos e use ferramentas como Apache Tika ou pdftotext para extrair texto.

2. **Indexação**: Escolha um motor de busca (elasticsearch, OpenSearch, Sphinx, Solr etc.) e crie índices de texto. Por exemplo, no Elasticsearch você definiria índices e mapeamentos, depois enviaria os textos coletados (JSON, HTML puro ou já processado) via API. Nutch e Ahmia usam Elasticsearch por padrão. Para código, rode o indexador do OpenGrok sobre os diretórios de código clonado. Isso fará parsing dos arquivos e criará os índices de busca de código.

3. **Organização de dados**: Armazene os arquivos ou banco de dados de forma hierárquica: por tipo de dado (ex.: /dados/codigo/, /dados/web/, /dados/pdf/) ou por fonte (ex.: /repos/github/projeto1/, /sites/foruns/). Mantenha metadados (titulos, URLs, datas). Certifique-se de reservar espaço: buscas de “milhões de arquivos” podem gerar dezenas ou centenas de GB. Use cluster distribuído (vários nós Elasticsearch) se chegar a ter terabytes.

4. **Consulta e interface**: Configure uma interface de busca para o usuário. Ferramentas como Kibana (para Elasticsearch), SearxNG (meta-busca customizada) ou até um simples frontend (por ex. Swagger UI de elastic) podem servir. Certifique-se de incluir filtros (ex.: por fonte, tipo de arquivo) e possíveis segmentações (documentos versus código, web versus Tor).

Exemplo de comandos: Para Nutch+Elasticsearch, por exemplo:
```
bin/nutch inject seed-list.txt       # injeta URLs iniciais
bin/nutch crawl local-crawl -depth 5 -topN 100   # rastreia e extrai dados
bin/nutch solrindex http://localhost:8983/nutch $CRAWL_DIR   # indexa em Solr/Elastic
```

Para OpenGrok:
```
git clone https://github.com/oracle/opengrok.git
# Configurar diretório de fonte (ex: /usr/local/src)
```

```
./OpenGrok index /usr/local/src
```

Estes são exemplos simples; cada ferramenta tem documentação. Em essência, você baixa ou obtém conteúdo (código, sites, docs), organiza em pastas, e executa o indexador correspondente (OpenGrok para código, Nutch/Elasticsearch para web, Ahmia para .onion).

Ferramentas adicionais: Para fóruns ou redes sociais específicas, use APIs (se disponível) ou raspagem (p. ex. [Scrapy](https://scrapy.org) ou BeautifulSoup). Para motores P2P (yaCy) instale múltiplos nós. Para crawlers Tor, o Ahmia já tem pipeline pronto (códigos em Python/Django usando Elastic). Para PDFs/documents, o Apache Tika pode ser integrado na cadeia de indexação, extraindo texto para o índice.

Resumo: Não existe “um único” buscador open source que cubra tudo; mas combinando ferramentas adequadas (OpenGrok para código, Nutch/Elastic para web, Ahmia para Tor, Searx/YaCy para complementar) você pode montar uma infraestrutura própria. Essas soluções são escaláveis e suportam milhões/bilhões de documentos: por exemplo, Nutch/Elastic pode lidar com petabytes (ver Common Crawl) e Mwmbl já indexa bilhões de páginas.

Fontes: Documentação e projetos oficiais dos sistemas acima.

Pensou por 1m 3s

Sim, esse novo relatório ajuda! Ele trouxe uma descoberta que considero especialmente interessante para o nosso projeto: o Mwmbl, um mecanismo open-source que constrói seu próprio índice da web, em vez de simplesmente depender dos resultados do Google ou do Bing.

Após conferir as fontes oficiais, eu acrescentaria o Mwmbl à nossa pesquisa, mas corrigiria algumas informações do relatório.
1. O que realmente acrescentamos

1. Mwmbl
Nova descoberta relevante
É um buscador independente com índice próprio e coleta distribuída por voluntários.

O projeto segue ativo em outubro de 2026 e, no dia 22, anunciou o recurso Seed Search, que combina seu índice com resultados externos para descobrir novas páginas e expandir sua cobertura.
GitHub+1

GitHub · Pesquisar

Por que é interessante? Podemos utilizá-lo como mais uma fonte de pesquisa, inclusive através do SearXNG, que já possui integração documentada com o Mwmbl.
SearXNG Documentation (2026.10.29+cff884f98)

Existe até uma API de busca, com franquia gratuita documentada, que pode ser integrada diretamente ao nosso agente pesquisador.
Ethical Web Search for Developers

Entretanto, não consegui confirmar a afirmação de que ele já indexa bilhões de páginas. A documentação informa que seu índice ainda é muito menor que o dos grandes buscadores. Em dezembro de 2023, a equipe havia comunicado mais de 100 milhões de páginas indexadas e a meta de alcançar um bilhão.
mwmbl blog+1

2. Apache Nutch
Confirmado para expansão
Seu desenvolvimento continua ativo: a versão 1.23 foi lançada em 8 de setembro de 2026.

É um candidato robusto para construir nosso próprio índice de fóruns, páginas técnicas, blogs e documentação.
Release+1

GitHub

3. OpenGrok
Complemento ao Zoekt
A documentação confirma pesquisa em diferentes formatos de código e sistemas de versionamento, navegação por referências e histórico.

Eu o manteria como ferramenta complementar para estudar projetos complexos, em vez de substituir o Zoekt como buscador principal de comandos e scripts.
GitHub+1

GitHub

2. Correções necessárias no relatório
Afirmação das outras IAsO que verifiquei
Mwmbl já indexa bilhões de páginas Não confirmado. O índice é próprio, mas ainda significativamente menor que o dos buscadores comerciais.
Common Crawl utiliza Apache Nutch Confirmado. O CCBot é baseado no Nutch, segundo a documentação

atual do Common Crawl.
Common Crawl
Elasticsearch é simplesmente Apache 2.0 e open-source Desatualizado. Partes do código voltaram a ter opção AGPLv3 em 2024, enquanto a distribuição padrão continua sob Elastic License 2.0.
Elastic
Sourcegraph oferece uma edição comunitária gratuita para implantar Não contaria com isso. A documentação atual apresenta Code Search nos planos Enterprise Starter e Enterprise.
Sourcegraph docs
Os comandos de instalação do Nutch estão prontos para uso Não. O exemplo de inject omite parâmetros e solrindex foi removido nas versões modernas.
Apache Software Foundation+1
./OpenGrok index instala e indexa o sistema Exemplo incompleto. O OpenGrok precisa de indexador, caminhos, configuração e interface. A imagem oficial Docker facilita esse processo.
GitHub

3. Como fica nossa seleção atualizada
ARQUITETURA CONSOLIDADA — OUTUBRO DE 2026
Busca Git Zoekt + Software Heritage + GitHub/GitLab
Busca web SearXNG + Mwmbl + Common Crawl
Crawler direcionado Crawl4AI + Scrapy
Crawler massivo Apache Nutch (quando necessário)
Documentos Apache Tika + Docling
Arquivamento ArchiveBox + armazenamento de objetos
Pesquisa local OpenSearch (BM25 + vetores)
Agente de IA GPT Researcher + RAGFlow, sujeitos a testes de integração
Fontes adicionais Wayback, YaCy, Ahmia e bitmagnet, sob demanda

Para comandos como display bgp peer, route-policy, interface gpon e parâmetros de RADIUS, a busca textual exata será tão importante quanto a inteligência semântica. O OpenSearch oferece as duas modalidades e permite combiná-las em pesquisas híbridas.
OpenSearch Documentation

4. A melhoria que eu considero mais importante
A maior oportunidade que apareceu ao combinar todos os relatórios é construir um sistema de pesquisa contínua com expansão automática do acervo:
Descobrir
Fontes globais
Coletar
Conteúdo relevante

Indexar
Base permanente
Investigar
IA + evidências

Novas descobertas alimentam próximas pesquisas

Por exemplo, ao encontrar um manual Huawei NE8000, o sistema poderia descobrir referências a outras versões do VRP, novas famílias de roteadores e comandos relacionados. Essas referências seriam adicionadas à fila de investigação, com controle de profundidade, relevância, origem e limites de coleta.

Assim, não dependeríamos de pesquisar manualmente cada fabricante ou protocolo. A biblioteca cresceria de forma direcionada e verificável.

Minha avaliação: este último relatório trouxe uma adição realmente útil, o Mwmbl, e confirmou o Nutch como opção forte para indexação própria em larga escala. Mas não mudou a escolha principal: Zoekt para código, SearXNG para descoberta externa e OpenSearch para pesquisar nosso próprio acervo.

Pode continuar trazendo as descobertas das outras IAs. Estamos conseguindo distinguir o que merece entrar no projeto do que apenas duplicaria funcionalidades, antes de fechar uma arquitetura definitiva.

```
Esta conversa está sendo útil até agora?
```

Anexo B.2 — Markdown colado (2).md

Origem: arquivo enviado pelo usuário; reprodução literal do conteúdo textual (sem edição).

```
coloquei outros chats para pesquisar vou colocar tudo que eles encontraram aqui.



"Busca Massiva em Repositórios Git (GitHub / GitLab)

Indexar o código público mundial do GitHub/GitLab em disco local exige petabytes de armazenamento. No entanto, existem motores de busca de código abertos e ferramentas de varredura global projetadas para minerar e indexar milhões de arquivos e repositórios:

- **Zoekt (por Google / Sourcegraph):** Mecanismo de busca de código aberto baseado em trigramas. É o motor por trás de grandes ferramentas de busca de código. Permite clonar milhares de repositórios e indexá-los localmente para buscas regex instantâneas em milissegundos.
- **GitHound:** Ferramenta de inteligência/OSINT que utiliza a API do GitHub e padrões de busca avançados (dorks) para varrer todo o conteúdo público do GitHub em tempo real, buscando termos específicos, arquivos de configuração, scripts e comandos.
- **GH Archive + Google BigQuery:** O repositório de eventos e código público do GitHub é totalmente espelhado e atualizado de hora em hora na nuvem pública. Com SQL no BigQuery, é possível fazer varreduras diretas em **bilhões de linhas de código, commits e arquivos** do GitHub em segundos.
- **Livegrep / Hound:** Motores de busca regex para grandes volumes de código Git auto-hospedados, ideais para subir em servidores locais e indexar repositórios baixados em massa.

## Indexadores Globais da Web, Tor, PDFs e Forúns (O "Olho de Deus")

Para varrer a web aberta, fóruns, documentos PDF, rede Tor e torrents sem se limitar apenas ao Google, a combinação de motores de busca descentralizados, crawlers e meta-buscadores abertos permite criar essa infraestrutura:

### 1. Meta-Buscadores e Agregadores Globais

- **SearXNG:** O meta-buscador open-source mais completo. Ele faz consultas simultâneas e anônimas em mais de 70 motores de busca, incluindo Google, Bing, DuckDuckGo, buscadores da Rede Tor (Ahmia), motores de torrent (PirateBay, 1337x), fóruns (Reddit) e repositórios acadêmicos. Possui API nativa em JSON para extração automatizada de dados.

### 2. Crawlers e Motores de Busca Próprios

- **YaCy:** Um motor de busca P2P totalmente descentralizado e open-source. Ele funciona como um "Google próprio": possui um crawler ativo que varre a web, lê páginas HTML, baixa e lê arquivos PDF, analisa fóruns e pode ser configurado para rastrear a rede Tor (.onion). Ele constrói um índice Search/Lucene local no seu próprio servidor com os gigabytes de dados coletados.
- **Apache Nutch + Elasticsearch:** A solução industrial open-source para criar um crawler web de escala massiva. O Nutch varre milhões de URLs na internet, baixa os conteúdos (PDFs, manuais, tutoriais, páginas) e indexa tudo diretamente no Elasticsearch ou Meilisearch para buscas instantâneas por palavras-chave.

### 3. Raspadores e Arquivadores de Documentos

- **ArchiveBox:** Sistema auto-hospedado para arquivamento da web. Você envia uma lista de URLs, feeds
```

RSS ou resultados de busca, e ele baixa automaticamente cópias estáticas em HTML, PDF, imagens e texto puro de fóruns, blogs e sites de documentação, armazenando e organizando tudo em disco local.
- **SpiderFoot:** Plataforma OSINT de automação de coleta de dados que varre mais de 100 fontes da web aberta, deep web, fóruns, pastebins e registros de rede para mapear informações específicas.

Arquitetura Recomendada para Documentação de Redes e Telecom

Para estruturar uma base de dados local de gigabytes com comandos CLI, firmwares e tutoriais de equipamentos (Huawei NE8000, Datacom, Cisco, OLTs ZTE/VSOL/Fiberhome/Intelbras, BGP, PBR, BNG/BRAS, etc.), a pilha mais eficiente combina:

```
[ SearXNG / Crawlee ] ──> [ ArchiveBox / Scrapy ] ──> [ Paperless-ngx ] ──> [ Motor RAG / Qdrant ]
 (Busca em Tor/Web)         (Download de PDFs/Web)     (OCR e Organização)     (Busca por IA/VETOR)

```

1. **Coleta Automatizada (`Crawlee` ou `Scrapy` + `SearXNG`):** Scripts em Python raspam fóruns especializados (Under-Linux, MikroTik Forum, Cisco Community, comunidades de telecom) e APIs do GitHub/GitLab baixando manuais em PDF, comandos CLI e scripts de configuração.
2. **Organização e OCR (`Paperless-ngx`):** Sistema open-source gerenciador de documentos. Ao salvar milhares de PDFs de manuais da Huawei, ZTE e Datacom em uma pasta, o Paperless executa OCR automático, extrai todo o texto dos comandos e manuais e indexa com busca por palavra-chave e tags organizadas por marca/modelo.
3. **Busca Semântica Local (`AnythingLLM` / `Open WebUI` + `Qdrant`):** Alimenta um banco de dados vetorial local com a pasta de arquivos coletados. Permite fazer perguntas diretas como *"Qual o comando de redundância de BNG no NE8000?"* ou *"Como configurar PBR em OLT VSOL?"* e obter a sintaxe exata extraída dos manuais e fóruns indexados."

"**Sim, existem sistemas open-source que se aproximam disso**, mas com limitações importantes de escala e praticidade. Não existe um único “Olho de Deus” que indexe *todo* o GitHub + *toda* a web (incluindo Tor e torrents) de forma completa e local para um usuário comum — o volume é de petabytes. O que existe são ferramentas boas que você combina para chegar perto, especialmente no seu caso (comandos, configs e tutoriais de equipamentos de rede: Huawei, Cisco, Datacom, OLT ZTE/VSOL/FiberHome/Intelbras, MikroTik, BGP, MPLS, PBR, SonicWall, Hillstone, redundância NE8000 etc.).

1. Busca em Git / GitHub / GitLab (código, configs, scripts)

Não dá para baixar e indexar “o Git inteiro” localmente de forma realista (são bilhões de arquivos e dezenas/centenas de TB). O que funciona:

Ferramenta	Tipo	O que faz	Observação
Zoekt	Open-source (Apache 2.0)	Indexação trigram ultra-rápida de código. Motor usado pelo Sourcegraph e pelo Exact Code Search do GitLab.	O melhor para self-host. Indexa milhares de repos em minutos/segundos por repo.
Sourcebot	Open-source (Docker)	Interface pronta em cima do Zoekt. Suporta GitHub + GitLab (público e privado).	Deploy com um docker compose. Bom para começar.
Livegrep	Open-source	Busca regex interativa em bases de código grandes (escala de GB).	Mais antigo, ainda útil.
OpenGrok	Open-source	Navegação + cross-reference de código.	Bom para projetos Java/ legado.
Ferramentas CLI	Open-source	gh search code, RepoScout, dorky, gh-search, gh-find	Usam a API do GitHub/GitLab sem indexar tudo localmente.

Prática recomendada:\\

Use a busca nativa do GitHub (já é excelente em repositórios públicos) + clone/indexe com Zoekt/Sourcebot apenas os repos relevantes (scripts MikroTik, configs Huawei, templates BGP/MPLS etc.). Muitos tutoriais e configs de rede estão em GitHub.

2. Busca na web inteira (fóruns, PDFs, sites, manuais, Tor, torrents)

Aqui o caminho é combinar meta-busca + crawler próprio + datasets públicos:

Ferramenta	Tipo	O que faz	Melhor para
SearXNG	Open-source (AGPL)	Meta-search: agrega Google, Bing, DuckDuckGo, Brave etc. Self-hosted, sem tracking.	Uso diário. Fácil de subir com Docker.
YaCy	Open-source (P2P)	Motor de busca completo com crawler próprio. Você indexa sites específicos e constrói índice local.	Crawl direcionado (forum.mikrotik.com, support.huawei.com, docs Cisco etc.).
Common Crawl	Dataset público	Bilhões de páginas já crawladas da web. Ferramentas como ccrawl-cli para buscar e extrair texto.	Pesquisa histórica e em larga escala sem crawlar você mesmo.
bitmagnet	Open-source (MIT)	Crawler DHT + indexador de torrents self-hosted.	Conteúdo em torrents (manuais, firmwares, dumps de fóruns antigos).
Crawlers Tor	Vários (ex.: baseados em TorSpider)	Crawl de .onion + indexação (Solr/Elasticsearch).	Conteúdo na rede Tor (cuidado com o que baixa).

Para **PDFs e manuais** (o grosso do material de equipamentos de rede):

- Baixe os PDFs oficiais e de fóruns.
- Indexe localmente com: **Open Semantic Search**, **duct**, DocFinder, Recoll, ou Elasticsearch + extração de texto (trafilatura/pdfminer).

Abordagem prática recomendada para o seu caso (rede)

1. **Comece com SearXNG** (meta-search) — já resolve 70-80% das buscas diárias de tutoriais, comandos e configurações.
2. **YaCy ou crawler direcionado** nos sites que realmente importam:
   - forum.mikrotik.com
   - support.huawei.com / enterprise.huawei.com
   - Sites de Datacom, ZTE, FiberHome, Intelbras, TP-Link enterprise
   - Reddit (r/networking, r/mikrotik etc.) e Stack Exchange
3. **GitHub/Zoekt** para scripts e configs reais (há muito material de MikroTik, BGP, MPLS, scripts Huawei).
4. **Manuais PDF** → pasta organizada + indexador local (Open Semantic Search ou duct). Isso vira sua “base de gigas bem organizada”.
5. Common Crawl + ccrawl-cli quando precisar de material antigo ou difícil de achar.
6. bitmagnet só se realmente precisar de material que circula em torrent.

Realidade de escala (“milhares de gigas”)

- Indexar **toda** a web ou todo o Git localmente exige infraestrutura séria (muitos TBs de disco, bastante RAM/CPU e banda).
- Para o seu caso (comandos e tutoriais de equipamentos de rede) **não precisa disso**. Uma base bem curada de algumas dezenas a poucas centenas de GB (manuais + fóruns importantes + repos relevantes) já resolve quase tudo que é difícil de achar no YouTube.
- Comece pequeno, organize por fabricante/protocolo (pastas: /huawei/olt/, /mikrotik/bgp/, /cisco/mpls/ etc.) e vá crescendo.

Quer que eu monte um guia mais detalhado de instalação (Docker) de SearXNG + Sourcebot/Zoekt + YaCy, ou um script/exemplo de como organizar e indexar uma pasta de manuais e configs de rede? Ou prefere focar primeiro em um fabricante específico (MikroTik, Huawei, etc.)?"

```
"# Busca em código-fonte e repositórios Git  \
\
Existem várias ferramentas open source para busca em código-fonte espalhado por repositórios Git/
GitLab/GitHub. Por exemplo, o Sourcegraph mantém o \*\*Zoekt\*\*, um motor de busca textual (incluindo
regex) otimizado para código-fonte. Zoekt permite buscas rápidas por trechos de código em múltiplos
repositórios grandes, combinando indexação por trigramas e análise sintática. Outro projeto popular é o
\*\*OpenGrok\*\* (originalmente da Sun/Oracle), um buscador e cross-reference engine em Java, que
entende vários formatos de código e sistemas de controle de versão (Git, Mercurial, SVN etc.).
Ferramentas inspiradas no antigo Google Code Search, como o \*\*livegrep\*\* (open source do Google),
também oferecem busca interativa por regex em bases de código de até gigabytes. Na prática, ferramentas
como Sourcegraph conseguem escalar para milhões de repositórios: por exemplo, anunciam “Search across
1M+ public repositories” e operação em milissegundos mesmo para 100–1M de repos. \
\
\## Crawlers e indexação em larga escala na Web  \
\
Para buscas abrangendo a Web inteira (fora de repositórios de código), são usados crawlers e motores de
busca distribuídos open source. Um exemplo clássico é o \*\*Apache Nutch\*\*, um crawler extensível e
escalável. A documentação do Apache descreve o Nutch como “um projeto de software de crawling web open
source altamente extensível e escalável”. O Nutch pode rodar em uma única máquina ou ganhar escala em
cluster Hadoop, rastreando sites e coletando páginas para indexação. Outro crawler notável é o
\*\*Heritrix\*\* (da Internet Archive), também open source, projetado para arquivar páginas web de
forma massiva. Depois que as páginas são coletadas, frameworks de busca como \*\*Apache Solr\*\* ou
\*\*OpenSearch/Elasticsearch\*\* entram em ação. O Solr é uma plataforma de busca distribuída (baseada
em Lucene) com busca full-text, facetas, clustering e suporte a documentos ricos (Word, PDF). Ele
permite replicação de índices para alta disponibilidade e tolerância a falhas. O Elasticsearch
(atualmente fonte aberta como OpenSearch) oferece um motor de busca distribuído semelhante, com APIs
REST/JSON, escalando por shards em clusters. Em conjunto, Nutch+Solr ou Nutch+Elasticsearch formam uma
base open source completa para indexar e pesquisar milhões (ou mais) de páginas da Web.\
\
\## Busca em redes Tor, torrent e outras fontes  \
\
Além da Web convencional, há buscas específicas em fontes “profundas” (deep web). Na rede Tor (.onion),
não há um buscador único oficial, mas existem projetos como o \*\*Ahmia\*\* (Tor) que indexam serviços
ocultos através de \*crawling\* de links internos e submissões de usuários. Eles descobrem .onions por
diretórios públicos e indicações em fóruns. Em sistemas de compartilhamento via torrent, há ferramentas
que agregam índices de rastreadores. Por exemplo, o projeto \*Torrent Search\* (licenciado GPLv2) é um
aplicativo multiplataforma que busca arquivos \*.torrent\* em diferentes sites via plugins. Ele permite
configurar vários trackers/plugins de busca. Integrações diretas com redes de torrent exigiriam
rastrear APIs ou DHTs dos trackers, mas projetos open source focam em agregação de sites de torrent. Em
resumo, não existe um único motor open source que “pesquise tudo (Google + Bing + Tor + torrents)”, mas
combina-se crawlers (como Nutch) configurados para usar Tor, plugins de torrent e agregadores de
resultados para cobrir essas fontes específicas.  \
\
\## Indexação de PDFs e documentos técnicos  \
\
Muitos documentos técnicos — tutoriais, manuais, datasheets — vêm em formato PDF ou Office. Ferramentas
de busca open source integram bibliotecas como o \*\*Apache Tika\*\* para extrair texto desses
arquivos. O Apache Tika “detecta e extrai metadados e texto de mais de mil tipos de arquivo (por
exemplo, PPT, XLS, PDF)”, unificando a interface de parsing. Solr e Elasticsearch usam o Tika por baixo
dos panos, permitindo busca full-text em PDFs, DOCs etc. De fato, a página do Solr destaca “rich
document (e.g. PDF) handling” e \*indexação em tempo real\*. Assim, um buscador próprio consegue
pesquisar dentro de conteúdos de PDF (como guias de MPLS, BGP, comandos de equipamentos) da mesma forma
que textos HTML comuns. Quanto a fóruns técnicos (por exemplo, posts em StackExchange ou fóruns de
redes), qualquer crawler web configurado poderá indexar as páginas HTML se estas forem acessíveis. Não
há um índice público único, mas Nutch ou Scrapy podem ser usados para rastrear e incluir tópicos de
fóruns nas buscas.  \
\
\## Desempenho e escalabilidade massiva  \
\
Escalar para \*milhões ou bilhões\* de documentos exige infraestrutura distribuída. Tecnologias como
Solr e Elasticsearch suportam clusters com shards e réplicas, permitindo lidar com petabytes de dados.
Por exemplo, o Sourcegraph demonstra busca em “1M+ repos” em milissegundos. Em outro caso, motores como
```

```
o Gigablast (outro projeto de busca open source) chegaram a indexar \~1 bilhão de páginas. Na prática, montar um índice próprio “de Deus” exigiria dezenas ou centenas de servidores: crawlers (Nutch/Heritrix) distribuídos para coletar URLs, e uma farm de servidores de busca (Solr/OpenSearch) para armazenar índices invertidos. Conforme citado, o Nutch foi projetado para clusters Hadoop, enquanto Solr foi feito para escalabilidade e tolerância a falhas. Com tal arquitetura, seria possível indexar todo tipo de conteúdo citado — Git, web pública, .onion, torrents, PDFs — em bases de dados com milhares de gigs de documentos. Em suma, existem projetos open source adequados (Zoekt, OpenGrok, Nutch, Solr, etc.) que, quando combinados e dimensionados adequadamente, cumprem o papel de um sistema de busca “universal” em redes e fóruns. \
\
\*\*Fontes:\*\* Projetos oficiais e documentação de Zoekt, OpenGrok, Sourcegraph, Apache Nutch, Apache Solr, Elasticsearch, Apache Tika, e descrições de ferramentas de torrent. Essas fontes descrevem capacidades e licenças dos sistemas citados."

Pensou por 1m 18s

Recebi os relatórios das outras IAs, incluindo os dois arquivos `.txt`. Eles acrescentaram ferramentas importantes que não tínhamos considerado inicialmente, principalmente Sourcebot, Apache Nutch, Apache Tika, Heritrix, OpenGrok, bitmagnet e Paperless-ngx.

A direção está ficando mais clara: precisamos de dois motores especializados — Git e Web — conectados a uma biblioteca permanente e a um agente de pesquisa profunda.

Antes de consolidar tudo, encontrei três correções importantes nas pesquisas:

1. GH Archive não é um espelho completo do código do GitHub. Ele armazena eventos públicos, como commits, issues e forks. Serve muito bem para descobrir atividade e repositórios, mas não permite pesquisar diretamente todas as linhas de código pelo BigQuery.&#x20;

   gharchive.org+1

2. Sourcebot deixou de ser totalmente open-source nas versões novas. Em 8 de outubro de 2026, o projeto anunciou que o desenvolvimento a partir da versão 6 seria fechado. As versões anteriores continuam disponíveis. Portanto, eu manteria o Zoekt como núcleo aberto de longo prazo.&#x20;

   Sourcebot+1

3. O Open Deep Research da LangChain foi arquivado em 21 de agosto de 2026. Ainda serve como referência de arquitetura, mas não seria minha primeira escolha para iniciar um projeto novo.&#x20;

   GitHub



Isso mostra por que não devemos simplesmente somar todas as recomendações das IAs. Algumas ferramentas são redundantes, outras mudaram de licença e algumas afirmações confundem descoberta de dados com indexação integral.

## Consolidação técnica: o que realmente vale a pena aproveitar

| Projeto                                                
```

```
Essencial                   | Identificação e extração de conteúdo de múltiplos formatos            |
| [Docling](https://github.com/docling-project/docling?utm_source=chatgpt.com)          |
Essencial                   | Extração estruturada de manuais, tabelas e PDFs                        |
| [GPT Researcher](https://github.com/assafelovic/gpt-researcher?utm_source=chatgpt.com) |
Essencial                   | Agente de pesquisa multietapas                                         |
| [RAGFlow](https://github.com/infiniflow/ragflow?utm_source=chatgpt.com)               | Alta
prioridade              | Biblioteca de conhecimento com IA                                            |
| [OpenSearch](https://github.com/opensearch-project/OpenSearch?utm_source=chatgpt.com)  | Alta
prioridade              | Índice textual e semântico de documentos                                     |
| [Apache Nutch](https://github.com/apache/nutch?utm_source=chatgpt.com)                | Segunda
fase                 | Rastreamento em massa e expansão distribuída                                 |
| [OpenGrok](https://github.com/oracle/opengrok?utm_source=chatgpt.com)                 |
Complementar                | Histórico e navegação entre símbolos do código                         |
| [Paperless-ngx](https://github.com/paperless-ngx/paperless-ngx?utm_source=chatgpt.com) |
Opcional                    | Excelente gerenciador documental, mas não obrigatório junto ao RAGFlow |
| [bitmagnet](https://github.com/bitmagnet-io/bitmagnet?utm_source=chatgpt.com)         | Terceira
fase                | Descoberta de metadados na rede BitTorrent                                   |
| [Ahmia Crawler](https://github.com/ahmia/ahmia-crawler?utm_source=chatgpt.com)        | Terceira
fase                | Pesquisa em serviços Tor públicos conhecidos                                 |

O RAGFlow merece atenção especial. Suas atualizações de 2026 acrescentaram recursos de pesquisa em múltiplas etapas, compilação de conhecimento e ingestão de conteúdo web. Isso o torna um candidato particularmente forte para o acervo documental.&#x20;

GitHub



Já o Apache Nutch continua bastante relevante para coleta em massa, inclusive com plugins para Apache Tika e mecanismos de indexação. Mas começar por ele adicionaria complexidade desnecessária antes de sabermos quais sites e fabricantes apresentam melhores resultados.&#x20;

Apache Nutch™



## Outras correções importantes

Software Heritage: o snapshot de junho de 2026 realmente registra aproximadamente 58,7 bilhões de nós e 1,1 trilhão de relações. Entretanto, os nós representam diferentes objetos do grafo de software; não são 58,7 bilhões de arquivos independentes. O grafo compactado ocupa aproximadamente 16 TiB.&#x20;

Software Heritage documentation



Zoekt e RAM: um relatório recomenda memória equivalente a 20% do corpus, mas a FAQ do Zoekt recomenda 20% a mais que o tamanho do corpus para o servidor de busca. Isso não é um limite absoluto, porque o motor usa arquivos mapeados em memória, porém significa que grandes índices podem sofrer bastante com falta de cache.&#x20;

GitHub+1



Common Crawl: ele não oferece uma pesquisa textual universal instantânea em bilhões de páginas. Seu índice público é principalmente orientado a URLs e registros WARC. Uma pesquisa por uma expressão rara exige localizar conjuntos candidatos e processar conteúdo, o que pode custar tempo e recursos.&#x20;

Common Crawl



SearXNG: centenas de integrações disponíveis não significam centenas de mecanismos operacionais simultâneos. Alguns podem retornar CAPTCHA, erros ou bloqueios, e a saída JSON precisa estar habilitada.&#x20;
```

SearXNG Documentation (2026.10.25+54ca1d9cf)+1

Além disso, eu retiraria GitHound e SpiderFoot do núcleo principal. São mais orientados a investigações de segurança e OSINT do que à construção de uma enciclopédia de comandos e protocolos. Também não trataria exemplos de VRRP, MESH e sincronização PPPoE como soluções equivalentes de redundância sem comprovação específica por plataforma.

A arquitetura provisória mais equilibrada

Agente pesquisador

GPT Researcher + ferramentas próprias

Motor Git

GitHub/GitLab, Software Heritage, ecosyste.ms, Zoekt

Motor Web

SearXNG, Common Crawl, Crawl4AI, Wayback

Processamento e preservação

Apache Tika, Docling, ArchiveBox, classificação, deduplicação e validação

Biblioteca permanente

Armazenamento de objetos + PostgreSQL + OpenSearch + RAGFlow

Conhecimento técnico verificável

Comandos CLI, fabricantes, firmware, protocolos, tutoriais, configurações, evidências e referências

Eu acrescentaria uma capacidade que nenhum dos relatórios explorou suficientemente: a validação técnica automatizada das descobertas.

O sistema não deveria apenas dizer que encontrou um comando Huawei ou Datacom. Ele precisaria distinguir se o comando é de visualização ou configuração, a família do equipamento, a versão do sistema operacional, os pré-requisitos e se existe comprovação em documentação oficial. Uma IA pode ajudar a classificar essas informações, mas não deve inventar compatibilidade nem executar comandos diretamente em equipamentos de produção.

Essa camada seria particularmente valiosa para utilizar futuramente o acervo nas integrações com OLTs, BRAS e demais equipamentos do NexoISP.

Minha avaliação até agora

Os outros relatórios melhoraram bastante a cobertura da pesquisa. No entanto, ainda não considero a arquitetura definitiva.

Faltam três decisões de engenharia importantes: comparar o RAGFlow com uma implementação própria sobre OpenSearch, estabelecer uma estratégia de descoberta contínua sem depender excessivamente de buscadores comerciais e definir como validar comandos de diferentes versões de firmware.

Também precisamos testar a qualidade real das pesquisas, não apenas o número de repositórios ou páginas anunciados pelos projetos.

O objetivo final não deve ser simplesmente armazenar 10 TB de arquivos. Deve ser encontrar uma informação técnica rara, correta e verificável em meio a esses 10 TB.

Os relatórios que você trouxe já oferecem uma boa base para isso.

Anexo B.3 — Texto colado (3).txt

Origem: arquivo enviado pelo usuário; reprodução literal do conteúdo textual (sem edição).

```
Arquitetura de Sistemas de Indexação Distribuída: Motores Open-Source para Varredura de Repositórios Git, Teia Global, Darknet e Redes P2P Aplicados à Engenharia de TelecomunicaçõesA expansão massiva da infraestrutura digital e a descentralização de ativos de software e documentação técnica criaram um cenário em que informações críticas de engenharia — tais como manuais de sintaxe CLI, arquivos MIB SNMP, guias de interoperabilidade de roteadores de borda e scripts de configuração — encontram-se dispersos por ecossistemas heterogêneos. Tais ativos estão alocados em milhões de repositórios Git, fóruns especializados, coleções de documentos PDF não indexados pelos motores de busca convencionais, redes ocultas da Darknet (serviços .onion) e enxames de compartilhamento P2P (Peer-to-Peer) via tabela hash distribuída (DHT).A viabilidade técnica de construir uma plataforma automatizada de agregação e indexação local — configurando uma base de conhecimento unificada de grande porte — depende da integração de duas categorias distintas de sistemas open-source: motores de busca de código-fonte orientados a sintaxe e compiladores de busca multicanal para a teia web aberta, profunda e distribuída.Motores de Busca e Indexação de Código em Larga Escala no Ecossistema GitPara pesquisar em milhões ou bilhões de linhas de código e arquivos de configuração distribuídos no GitHub, GitLab, Gerrit ou servidores Git privados, motores tradicionais baseados em varredura linear (grep) mostram-se ineficientes devido à alta latência de entrada e saída (I/O). A solução arquitetural para varrer o ecossistema Git exige motores de busca baseados em indexação de trigramas posicionais e análise de sintaxe abstrata.Zoekt: Indexação por Trigramas e Espelhamento MassivoO Zoekt é uma das ferramentas open-source mais robustas para busca de código-fonte em escala gigabyte e terabyte. Desenvolvido originalmente no âmbito da Google e mantido ativamente pelo projeto Sourcegraph, o Zoekt atua como o motor primário de busca textual de alta velocidade da plataforma Sourcegraph.Parâmetro / RecursoDescrição Técnica no ZoektMecanismo de IndexaçãoTrigramas posicionais UTF-8 que dividem o código em grupos de 3 caracteres unicode e armazenam os deslocamentos (offsets).Desempenho de ConsultaLatência típica na ordem de ~10 ms para termos raros em caches quentes do sistema operacional.Ranqueamento de SímbolosIntegração nativa com ctags para identificar e elevar declarações de funções e variáveis no topo dos resultados.Espelhamento de GitUtilitários internos (zoekt-git-index, zoekt-github-getly) que clonam e mantêm atualizados repositórios de organizações inteiras.Indexação MultiblocoCapacidade de indexar múltiplas ramificações (branches) concorrentemente sem duplicar dados inalterados.A taxa de expansão do índice do Zoekt demanda recursos computacionais bem definidos. O tamanho do arquivo de índice gerado ($S_{idx}$) guarda uma relação proporcional direta com o tamanho do corpus de código bruto ($S_{corpus}$), expressa pela equação:$$S_{idx} \approx 3{,}5 \times S_{corpus}$$Para manter latências de busca sub-segundo, o Zoekt mapeia os arquivos de índice diretamente na memória RAM via chamada de sistema mmap. Recomenda-se que o servidor disponha de um volume de memória RAM correspondente a pelo menos $20\%$ do tamanho total do corpus, além de discos de estado sólido (SSD/NVMe) dedicados.Livegrep, Hound, OpenGrok e RepowiseAlém do Zoekt, existem alternativas open-source com focos específicos de implantação:O Livegrep, projetado com base no Google Code Search, executa buscas regex interativas sobre repositórios na escala de gigabytes. Utiliza um modelo em que o processo indexador (codesearch) gera arquivos .idx independentes, comunicando-se via gRPC com a interface web (livegrep). Embora forneça filtragem em tempo real enquanto o usuário digita, apresenta consumo elevado de RAM e CPU durante a fase de compilação do índice e carece de pontuação avançada por símbolos.O Hound, desenvolvido originalmente pela Etsy, utiliza um índice de $n$-gramas escrito em Go para varrer repositórios Git via expressões regulares. Possui arquitetura simples de configurar, porém entrega desempenho cerca de dez vezes inferior ao Zoekt em grandes volumes de dados.O OpenGrok é um motor tradicional sob licença CDDL, focado em navegação de código, referências cruzadas e histórico de sistemas de controle de versão (VCS). É amplamente utilizado em ecossistemas Java enterprise, permitindo rastrear o histórico de commits e alterações ao longo do tempo.O Repowise é uma solução open-source moderna (AGPL-3.0) que combina busca literal por identificadores com análise semântica e rastreamento de árvore de sintaxe abstrata (AST). Permite interfacear com agentes de IA via protocolo MCP (Model Context Protocol), fornecendo contexto de dependências e decisões de arquitetura.Motor de BuscaLicençaAlgoritmo de IndexaçãoSuporte a Regex e ExpressõesRanqueamento de SímbolosEspelhamento Autônomo do GitZoektApache 2.0Trigramas PosicionaisSim (Sintaxe RE2)Sim (via ctags)Sim (zoekt-github-index)LivegrepOpen SourceN-gramas MapeadosSim (Interativo)RudimentarParcial (via scripts auxiliares)HoundMITN-gramas GoSimNãoSim (via config.json)OpenGrokCDDL 1.0Lucene / Inverted IndexSimSim (Cross-Reference)Sim (Integração VCS)RepowiseAGPL-3.0Híbrido (AST + Vetorial)Identificadores/ LiteraisSim (Contexto de Nós)Sim (Git local/remoto)Plataformas comerciais fechadas, como o GitHub Code Search (desenvolvido internamente em Rust) e o Sourcegraph Enterprise, utilizam conceitos semelhantes de indexação por trigramas e análise sintática, porém limitam o acesso ao código-fonte do motor de busca.Varredura Multicanal: Web Aberta, Profunda, Redes Tor e Enxames BitTorrent (DHT)Para expandir a coleta de dados além dos repositórios de código, alcançando manuais em PDF, drivers legados, arquivos de configuração expostos e discussões em fóruns, é necessário empregar motores de metabusca, crawlers de teia web profunda e indexadores P2P.Metabusca e Rastreamento Web Aberto: SearXNG e Apache NutchO SearXNG é um motor de metabusca free and open-source (AGPL-3.0) que atua como agregador anônimo. Ele
```

intercepta a consulta do usuário e a distribui concorrentemente para mais de 250 serviços upstream — incluindo Google, Bing, DuckDuckGo, Qwant, bases de documentos e motores especializados. O SearXNG expõe uma API em formato JSON e aceita "bangs" (sintaxe iniciada por !), permitindo isolar consultas em motores específicos sem armazenar perfis ou históricos de navegação.Para a extração profunda de arquivos brutos (como manuais em PDF e instaladores de firmware expostos em diretórios abertos), utiliza-se o Apache Nutch. O Nutch é um web crawler altamente escalável e modular que realiza a varredura, raspagem, extração de texto em documentos binários (via integração com Apache Tika) e exportação estruturada para bancos de dados ou motores de busca como Elasticsearch e Solr.Rastreamento da Darknet e Tor (.onion)A navegação e extração de dados na rede Tor para a captura de manuais vazados, configurações de equipamentos de telecomunicações e postagens em fóruns restritos exigem ferramentas preparadas para operar sobre proxies SOCKS5 (127.0.0.1:9050).O OWASP TorBot, desenvolvido em Python sob os auspícios da OWASP, é voltado para investigação de inteligência em fontes abertas (OSINT) na Dark Web. Ele executa algoritmos de varredura por busca em largura (BFS) a partir de URLs semente (seed URLs), verifica a disponibilidade operacional dos serviços ocultos (live check), coleta metadados de cabeçalho HTTP, extrai títulos, captura estruturas HTML e gera árvores de relacionamentos de links em formato JSON. O TorBot integra-se nativamente a motores de busca focados na rede Tor, como o Ahmia.fi e o OnionSearch.O TorCrawl.py é um script em Python focado em raspagem anônima e extração de dados. Permite a navegação profunda através da rede Tor com controle rigoroso do tempo de pausa entre requisições (--pause), profundidade de navegação (--depth) e rotação de cabeçalhos User-Agent. O TorCrawl.py possui suporte à filtragem por regras YARA, viabilizando a identificação automatizada de palavras-chave estruturadas em arquivos brutos baixados durante o processo de varredura.Rastreamento da Tabela Hash Distribuída (DHT) e BitTorrentUma parcela significativa de backups de documentação industrial, imagens de disco e coleções de firmware transita exclusivamente via redes P2P. A indexação dessas informações independe de rastreadores (trackers) centrais quando executada por meio de crawlers de DHT.O bitmagnet é uma solução open-source auto-hospedada (desenvolvida em Go e TypeScript) que funciona como indexador BitTorrent, crawler de DHT e motor de busca com interface web e API GraphQL. O bitmagnet escuta passivamente as mensagens de anúncio de nós na rede DHT de Mainline (Kademlia), identifica os infohashes em circulação e realiza a requisição das metadados do torrent conforme especificado pelos protocolos BEP-0009 e BEP-0010.Esse processo permite mapear o nome da coleção, tamanho total, lista completa de arquivos internos (incluindo extensões .pdf, .bin, .iso ou .zip) sem a necessidade de realizar o download do conteúdo dos arquivos brutos. Outros projetos similares incluem o dhtcrawler2 (escrito em Erlang) e o dht-crawler.A integração arquitetural opera mediante um fluxo contínuo em que as fontes de dados heterogêneas (repositórios Git, sites web, serviços ocultos .onion e enxames DHT) são varridas respectivamente pelos motores especializados (Zoekt, SearXNG/Nutch, TorBot/TorCrawl e bitmagnet). Os dados brutos resultantes são canalizados para um pipeline centralizado de extração, normalização e categorização sintática, sendo finalmente alimentados em um banco de dados local indexado com suporte a buscas de alta velocidade.Aplicação Prática: Construção da Base de Conhecimento "Olho de Deus" para Engenharia de RedesA combinação dessas ferramentas permite automatizar a criação de um repositório técnico local estruturado em gigabytes ou terabytes. Esse repositório organiza comandos CLI raros, scripts de automação, topologias e manuais de equipamentos de diversos fabricantes.Escopo de Cobertura por Fabricante e TecnologiasA integração dos coletores permite indexar especificações e sintaxes operacionais de hardware de múltiplos provedores de serviço (ISPs) e operadoras:O ecossistema Huawei engloba configurações de borda para roteadores da série NetEngine (NE8000 F1A, F8), incluindo comandos para redundância BGP, famílias de endereços L2VPN/VPNv4 e sincronização de sessões BGP. Inclui também a configuração avançada de OLTs GPON/XGS-PON (ex.: SmartAX MA5800, EA5800) e provisionamento de ONUs.Para o fabricante Datacom, a cobertura abrange dispositivos executando o sistema operacional DmOS. Há coleta automatizada de manuais, guias rápidos de configuração e exportação de MIBs SNMP via TFTP/SCP, bem como a extração de modelos oficiais de monitoramento Zabbix e painéis Grafana disponibilizados em repositórios Git públicos (datacom-teracom/dmos-zabbix-template).Em relação a ZTE, Fiberhome, VSOL, Intelbras, Digistar, TP-Link e Furukawa, a varredura engloba scripting CLI, sintaxes de ativação de perfis de linha VLAN, modos de operação bridge/router em OLTs e ONUs, e gerenciamento via MIBs proprietárias.Para Cisco e Mikrotik, indexam-se rotinas de configuração avançada de Mikrotik RouterOS como autenticador concentrador (PPPoE/RADIUS) com redundância via VRRP e MESH, além de roteamento dinâmico.No domínio de Segurança e Roteamento Avançado, capturam-se tabelas de decisão para Policy-Based Routing (PBR), engenharia de tráfego BGP (Local Preference, AS-Path Prepending, Communities), VPNs MPLS L2/L3 e regras de inspeção para firewalls de próxima geração (NGFW) como SonicWall e Hillstone E-Series.Pipeline de Extração e Organização de ArquivosPara transformar o fluxo contínuo de dados não estruturados em uma base acessível, adota-se um pipeline contínuo dividido em quatro estágios sequenciais.No primeiro estágio, referente à Ingestão Filtrada, o Zoekt clona repositórios públicos contendo dotfiles, scripts em Python/Ansible e documentações em Markdown. Simultaneamente, o SearXNG e o Apache Nutch realizam varreduras filtradas por operadores do tipo filetype:pdf "NE8000" "BGP" ou filetype:txt "display dmos".No segundo estágio, dedicado à Conversão e Parsing de Documentos, os documentos binários (PDFs de manuais técnicos) passam por extratores de texto brutos que os convertem em arquivos .md ou .txt estruturados. Tabelas de comandos CLI de fabricantes como Datacom e Hillstone são preservadas em texto simples.No terceiro estágio, focado na Categorização por Expressões Regulares e Marcações, um script Python pós-processador varre os arquivos salvos aplicando filtros de padrões técnicos para identificar assinaturas específicas

```
de sintaxe (como display bgp routing-table para Huawei NE8000, interface gpon para Datacom DmOS, /ppp
secret para Mikrotik e route-policy para SonicWall PBR).No quarto estágio, correspondente ao
Armazenamento Estruturado e Indexação Local, os arquivos limpos são gravados em uma estrutura
hierárquica no sistema de arquivos local organizada por fabricante, modelo e protocolo (por exemplo,
divididos em diretórios dedicados a /huawei/ne8000/, /datacom_dmos/mibs/, /mikrotik/ e /firewalls/
sonicwall/). Uma instância local do Zoekt é apontada diretamente para a raiz dessa árvore de
diretórios, permitindo consultas instantâneas por expressões regulares.Comparativo Integrado das
Soluções de Indexação e ColetaA tabela abaixo sintetiza a adequação das ferramentas apresentadas de
acordo com as camadas do ecossistema de dados:CategoriaSistema Open-SourceEscopo de ColetaVantagem
Arquitetural PrimáriaRequisito de InfraestruturaBusca de CódigoZoektRepositórios Git (GitHub, GitLab,
Gerrit)Trigramas posicionais e suporte nativo a ctagsSSD NVMe rápido; RAM $\ge 20\%$ do corpusBusca de
CódigoLivegrepCódigo-fonte local e repositórios mapeadosBusca regex interativa com resposta ao
digitarMapeamento intensivo de memória RAMMetabusca WebSearXNG$+250$ motores web (Google, Bing,
DDG)Agregação sem rastreamento com saída JSONBaixo consumo computacional (Python)Rastreamento WebApache
NutchWebsites abertos, diretórios e PDFsArquitetura distribuída e extensível via pluginsCluster Hadoop
ou nós Java de alta capacidadeDark Web (OSINT)OWASP TorBotDomínios .onion da rede TorMapeamento de
links em árvore e live checkInstância Tor local (proxy SOCKS5)Dark Web (Scraper)TorCrawl.pyPáginas
brancas e ocultas em TorFiltragem de conteúdo em tempo real via YARAPython 3 com serviço Tor
ativoIndexação P2PbitmagnetEnxames BitTorrent e rede DHTCaptura metadados sem baixar arquivos
brutosBanco de dados PostgreSQL + suporte a DockerDesafios Técnicos, Operacionais e LimitaçõesA
implementação de uma arquitetura centralizada de coleta e indexação em larga escala impõe desafios de
ordem prática.O consumo de largura de banda e os bloqueios por taxa (rate limiting) representam o
primeiro obstáculo significativo. O uso do SearXNG e do Apache Nutch em frequência elevada sem
gerenciamento de proxies pode resultar no bloqueio de endereços IP por parte dos motores upstream e de
redes de entrega de conteúdo (CDNs). É necessário configurar rotação de IPs e respeitar delays entre
requisições.A volatilidade da Darknet constitui outro fator crítico. Serviços .onion possuem taxas
elevadas de indisponibilidade temporária ou definitiva. O TorBot minimiza requisições falhas executando
checagens de estado (live checks) antes de iniciar o processo de extração profunda.A segurança e o
isolamento de bordo (OpSec) exigem atenção rigorosa. O rastreamento de arquivos executáveis, firmwares
não verificados ou scripts provenientes de fontes desconhecidas ou redes P2P exige que o ambiente de
coleta e processamento funcione de forma totalmente isolada (containers Docker ou máquinas virtuais
segmentadas), sem acesso direto a sistemas de produção.Por fim, o gerenciamento do volume de dados
requer estratégias rígidas de filtragem. A indexação de redes DHT pode gerar volumes massivos de
metadados em curtos intervalos de tempo. Recomenda-se aplicar filtros rígidos de extensão de arquivo
(limitando a captura a extensões como .pdf, .txt, .bin, .iso, .zip, .py, .clp) diretamente no nível do
pipeline de ingestão.ConclusãoÉ perfeitamente viável construir um sistema centralizado e automatizado
para indexação global e busca local de dados técnicos baseando-se exclusivamente em soluções open-
source.Para a camada de código-fonte e repositórios Git, o Zoekt estabelece-se como o motor mais
eficiente, garantindo buscas sub-segundo via indexação por trigramas e suporte nativo ao espelhamento
de organizações do GitHub e GitLab. Para a camada de busca externa, a integração do SearXNG (metabusca
aberta), Apache Nutch (raspagem de documentos), OWASP TorBot / TorCrawl.py (varredura da rede Tor) e
bitmagnet (mapeamento de arquivos na rede DHT) provê cobertura completa sobre dados não indexados pelos
motores de busca convencionais.A integração dessas ferramentas em um pipeline unificado de extração,
conversão e categorização viabiliza a criação de um repositório local robusto. Esse repositório reduz
dramaticamente o tempo de diagnóstico e implementação de configurações complexas em redes multivendor
(Huawei, Datacom, Cisco, Mikrotik e outros), consolidando um acervo técnico especializado e de alta
disponibilidade.
```

Anexo B.4 — Texto colado (4).txt

Origem: arquivo enviado pelo usuário; reprodução literal do conteúdo textual (sem edição).

```
# Sistemas open source para pesquisar “o Git inteiro” e construir um “olho de Deus” de conhecimento
técnico

## O que realmente existe hoje

Sim, existem sistemas open source que chegam **muito perto da primeira ideia**: pesquisar quantidades
gigantescas de código provenientes de GitHub, GitLab, Bitbucket, forges independentes e pacotes de
software. O projeto que mais se aproxima de uma “biblioteca universal de código-fonte” não é o GitHub
Search: é o **Software Heritage**. A missão declarada do projeto é coletar e preservar todo software-
fonte publicamente disponível, vindo de GitHub, GitLab, Bitbucket, servidores Git, SVN, Mercurial,
Debian, PyPI, npm, CRAN e outras fontes. 〖cite turn808118search3 turn808118search4〗
```

A escala já é exatamente da ordem que você imaginou. O snapshot público do Software Heritage de junho de 2026 possui aproximadamente **58,7 bilhões de nós e 1,1 trilhão de relações** no grafo de software. Somente a representação compactada desse grafo ocupa aproximadamente **16 TiB**, enquanto a versão tabular em Apache ORC ocupa aproximadamente **33 TiB**. Isso é antes de imaginar uma cópia completa de todos os conteúdos propriamente ditos. 〖cite turn287925search0〗 Em seu relatório de atividade de 2025, o projeto informou já preservar cerca de **27 bilhões de arquivos**; no relatório anterior eram mais de 22 bilhões de arquivos únicos e 340 milhões de projetos. 〖cite turn808118search5〗

Portanto, para a pergunta “existe alguma coisa que pesquise milhões ou bilhões de informações em Git?” a resposta é **sim**. Para “existe um programa que eu instalo e imediatamente tenho todo o Git público do planeta pesquisável como Google?”, a resposta é **não em uma única aplicação pronta**. O caminho real é combinar uma fonte global de dados como Software Heritage com um mecanismo de indexação próprio. O Software Heritage fornece o acervo e datasets gigantescos; ferramentas como **Zoekt**, **OpenGrok** ou **Livegrep** fazem a pesquisa extremamente rápida sobre o subconjunto que você decidiu indexar. 〖cite turn287925search1 turn808118search2 turn721123search0 turn721123search3〗

Há ainda o **World of Code**, criado especificamente para mineração em escala de ecossistemas inteiros de software open source. Ele agrega milhões de repositórios de plataformas como GitHub e GitLab e mantém ligações entre projetos, commits, arquivos e autores. É especialmente interessante para investigação e mineração de software, embora seja menos parecido com um “Google de arquivos” pronto para uso do que Zoekt/OpenGrok. 〖cite turn287925search3 turn287925search4〗

Um detalhe importante: confiar exclusivamente na busca do GitHub não produz o corpus completo. A própria documentação do GitHub informa restrições de indexação: somente determinados arquivos e repositórios entram na pesquisa, branches não-default normalmente ficam de fora, arquivos grandes têm limites, repositórios arquivados não são pesquisáveis e a API REST de busca retorna no máximo 1.000 resultados por consulta, além de possuir limites específicos de taxa. 〖cite turn287925search5 turn287925search6〗

A pilha que eu usaria para o “Google de Git”

Para sua finalidade, eu usaria **Software Heritage + Zoekt**, não tentaria duplicar o GitHub inteiro usando `git clone`.

O Software Heritage seria sua camada de descoberta histórica/global; o **Zoekt** seria o mecanismo de pesquisa textual ultrarrápida do material selecionado. Zoekt utiliza índices baseados em trigramas, suporta substring, expressões regulares, operadores booleanos e filtros por arquivo/repositório, e foi feito justamente para pesquisar grandes conjuntos de código com latência muito menor que um `grep` convencional. Ele também possui ferramentas para espelhar código de hosts Git comuns. 〖cite turn808118search1 turn808118search2〗

Se você valorizar navegação por símbolos, histórico e referências cruzadas — por exemplo, encontrar não apenas a string `display bgp peer`, mas em qual arquivo, módulo, classe ou versão ela apareceu — **OpenGrok** é excelente. O projeto é um mecanismo open source de busca e cross-reference para árvores de código, compreende diversos formatos e históricos de sistemas de controle de versão. 〖cite turn721123search0 turn721123search2〗

Livegrep é ainda mais simples e agressivamente rápido para regex e strings. Ele cria índices persistentes, pode trabalhar com índices maiores que a RAM por meio de memory mapping e documenta que seus índices costumam ocupar algo como três a cinco vezes o tamanho do texto indexado. Por essa característica, eu o escolheria para um corpus menor e altamente consultado, e não como índice principal de dezenas de terabytes. 〖cite turn721123search3〗

O equivalente para “a web inteira”

Para sua segunda ideia, não encontrei um único projeto open source que faça sozinho, de maneira completa, **Google + Bing + DuckDuckGo + fóruns + PDFs + arquivos + sites históricos + Tor + BitTorrent**. O sistema que você está imaginando existe tecnicamente, mas precisa ser construído como uma **federação de coletores e índices**.

A composição que mais se aproxima seria:

Camada	Projeto mais indicado	Papel
Metabusca pública	**SearXNG**	Consulta simultaneamente centenas de mecanismos/fontes
Corpus massivo já coletado	**Common Crawl**	Fornece bilhões de páginas em WARC/WAT/WET

Seu próprio crawler	**Apache Nutch** ou **Heritrix**	Varre sites e cria seu próprio acervo
Sites modernos/JavaScript	**Crawl4AI**	Converte páginas em conteúdo limpo para IA/RAG
PDF, DOC, PPT, XLS etc.	**Apache Tika**	Extrai texto e metadados
Arquivo histórico	**Internet Archive/Wayback CDX**	Descobre capturas antigas
Tor	**Ahmia crawler/index**	Indexa serviços `.onion` descobertos
BitTorrent	**bitmagnet**	Crawler da DHT e índice de metadados de torrents
Pesquisa local	**OpenSearch**	BM25 + busca semântica/vetorial
Agente pesquisador	**GPT Researcher**	Formula consultas, lê resultados e cria relatórios

O **SearXNG** é particularmente útil como “radar”. Em outubro de 2026 sua documentação informa suporte à agregação de resultados de até aproximadamente **260 serviços de busca**; ele é self-hosted e pode encaminhar pesquisas através de Tor. Porém é fundamental entender que SearXNG **não possui uma cópia da web**: ele é um metabuscardor, enviando sua consulta a outros motores e consolidando os resultados. 〖cite turn689589search0 turn689589search1 turn689589search2〗

Para não depender somente do que Google/Bing/DDG decidem retornar, a peça decisiva é o **Common Crawl**. O corpus contém petabytes de páginas coletadas desde 2008; a organização informa mais de **10 PB de conteúdo acumulado**, com crawls aproximadamente mensais e normalmente mais de **dois bilhões de páginas por ciclo**. Os dados podem ser acessados gratuitamente e são disponibilizados como WARC, WAT e WET. 〖cite turn808118search9 turn808118search10 turn808118search11〗

Isso muda completamente o projeto: em vez de perguntar “Google, me dê os dez resultados que você considera melhores”, você pode consultar diretamente um corpus de bilhões de páginas.

Para páginas que ainda não estejam no Common Crawl ou para sites que você queira acompanhar continuamente, **Apache Nutch** é uma das melhores opções. É um crawler maduro e escalável baseado em Hadoop, possui plugins de parsing e consegue enviar conteúdo diretamente para Solr ou Elasticsearch/OpenSearch. A própria documentação possui inclusive exemplos para pesquisa de arquivos Microsoft Office e PDF. 〖cite turn721123search5 turn721123search7〗

Outra opção é o **Heritrix**, crawler open source do Internet Archive. Ele foi projetado especificamente como crawler “web-scale” de qualidade arquivística e grava coleções de páginas que depois podem ser preservadas e reprocessadas. 〖cite turn721123search8〗

Para sites modernos carregados por JavaScript, **Crawl4AI** é uma camada complementar muito boa. Ele transforma páginas em Markdown e conteúdo estruturado apropriado para RAG e agentes de IA. A versão open source pode ser executada localmente; seu modo self-hosted faz crawling, enquanto as funções de busca global oferecidas pelo serviço hospedado são separadas. 〖cite turn205491search7 turn205491search8〗

E para aqueles PDFs estranhos, manuais antigos, Word, PowerPoint, planilhas, CHM, arquivos WARC etc., o componente certo é **Apache Tika**. A documentação atual informa suporte de detecção/extracção de texto e metadados para mais de mil tipos de arquivo, inclusive PDF, Office, HTML, RSS/Atom e arquivos web WARC/WACZ. 〖cite turn689589search4 turn689589search5〗

Tor, torrents e fontes que os buscadores convencionais não veem

A parte Tor é possível, mas aqui existe uma limitação estrutural importante: **não existe uma lista completa de todos os `.onion` existentes**. Serviços onion não funcionam como a DNS pública tradicional; para acessar um serviço você normalmente precisa conhecer seu endereço de 56 caracteres, e o próprio Tor Project explica que esses endereços não são indexados pelos mecanismos comuns da mesma maneira que sites tradicionais. 〖cite turn942089search0 turn942089search2〗

Por isso, nenhum crawler pode afirmar honestamente que pesquisou “100% da Tor”. Ele descobre serviços por sementes conhecidas, hyperlinks, diretórios, Onion-Location e outras fontes públicas, e então segue os links encontrados. O protocolo onion também permite serviços que exigem autorização do cliente, que obviamente não podem ser indexados por um crawler público sem as credenciais apropriadas. 〖cite turn942089search3 turn942089search4〗

O projeto open source mais interessante que encontrei para isso é **Ahmia**. O próprio projeto informa que qualquer pessoa pode montar seu próprio índice e crawler; a arquitetura é dividida em **ahmia-crawler**, **ahmia-index** e **ahmia-site**, e o crawler usa Scrapy sobre Tor para percorrer serviços onion conhecidos e descobrir outros links. 〖cite turn490676search0 turn490676search1 turn490676search2〗 A versão atual do crawler possui código específico para URLs `.onion` v3 e segue links encontrados nas páginas. 〖cite turn490676search3〗

BitTorrent é tecnicamente ainda mais interessante para sua ideia porque existe um mecanismo de

descoberta distribuído. **bitmagnet** é um indexador BitTorrent self-hosted que implementa crawler da **DHT**, coleta infohashes e pede os metadados aos peers, formando seu próprio mecanismo pesquisável sem depender de Pirate Bay, trackers ou indexadores externos. Ele oferece interface web, API GraphQL e pesquisa local. 〖cite turn490676search4 turn490676search5〗

O ancestral famoso dessa abordagem é **magnetico**, também um crawler/autonomous search engine da DHT BitTorrent. Entretanto o projeto original foi arquivado e tornou-se somente leitura em janeiro de 2022, portanto atualmente eu escolheria bitmagnet em vez dele. 〖cite turn490676search6〗

Aqui existe uma diferença crucial: indexar metadados públicos de torrents ou páginas públicas é uma coisa; baixar ou redistribuir conteúdo protegido, ilegal ou privado é outra. Um sistema sério deve armazenar URL, hash, título, metadados e texto permitido, aplicar políticas de retenção e evitar transformar automaticamente cada resultado encontrado em download.

Como eu montaria o seu sistema para telecom e redes

Para o caso que você descreveu — **Huawei, Cisco, ZTE, Datacom, FiberHome, VSOL, Intelbras, Digistar, TP-Link, Mikrotik, SonicWall, Hillstone, BGP, MPLS, PBR, redundância de BRAS/autenticadores, NE8000 etc.** — eu não tentaria baixar “toda a internet”. Isso produziria petabytes de lixo e tornaria o sinal técnico pior. Minha recomendação é um sistema com **cobertura extremamente ampla + coleta focada por assunto**. Essa é uma inferência de arquitetura baseada na diferença de escala entre Common Crawl/ Software Heritage e mecanismos de índice local. 〖cite turn808118search10 turn287925search0〗

Eu o dividiria conceitualmente assim:

```text
                        ┌──────────────────────────┐
                        │ Pergunta do usuário      │
                        │ "NE8000 BGP + Radius"    │
                        └────────────┬─────────────┘
                                     │
                                Research Agent
                              GPT Researcher / custom
                                     │
          ┌──────────────────────────┼──────────────────────────┐
          │                          │                          │
     CODE SEARCH                WEB SEARCH                LOCAL KB
  Software Heritage             SearXNG                 OpenSearch
  GitHub/GitLab APIs        Google/Bing/DDG/etc.      BM25 + vectors
  Zoekt/OpenGrok                  │                          │
          │                  ┌────┼────┐                     │
          │                  │    │    │                     │
       Git repos          Common Nutch Wayback             PDFs
                          Crawl    │                       DOCs
                                 Crawl4AI                  HTML
                                                            │
                                                          Tika
          │                                                 │
          └─────────────────────────┬───────────────────────┘
                                    │
                            Normalização / dedupe
                                    │
                  ┌─────────────────┼─────────────────┐
                  │                 │                 │
               MinIO           PostgreSQL        OpenSearch
          arquivo original      metadata        texto/índice
                  │
                  └─────────────────┬─────────────────┘
                                    │
                               RAG / pesquisa
                                    │
                              LLM local/cloud
```

Para busca, eu usaria **OpenSearch híbrido**: pesquisa lexical BM25 para comandos exatos e pesquisa vetorial/semântica para perguntas conceituais. Isso é especialmente apropriado para telecom porque `display bgp peer`, `peer x.x.x.x route-policy`, `ip policy route-map`, nomes de MIBs, mensagens de

erro e comandos CLI precisam de correspondência lexical precisa; já perguntas como “como fazer redundância de dois autenticadores PPPoE com BGP?” se beneficiam de correspondência semântica. OpenSearch implementa BM25 por padrão e também oferece hybrid search que combina consulta lexical e neural. 〖cite turn689589search6 turn689589search7〗

Eu preservaria também **o arquivo original**, não somente chunks vetorizados. Cada resultado deveria conter fabricante, família, modelo, versão de firmware/VRP/IOS, linguagem, tipo de documento, URL de origem, data capturada, hash SHA-256, primeira/última captura, título, comandos identificados e o arquivo bruto. Assim, uma resposta sobre um `NE8000 M8` não acaba misturando um exemplo escrito para outra geração ou outro firmware.

Uma estrutura útil poderia ficar assim:

```text
/base
 ├── code/
 │    ├── software-heritage/
 │    ├── github/
 │    ├── gitlab/
 │    └── vendor-tools/
 │
 ├── vendors/
 │    ├── huawei/
 │    │    ├── ne8000/
 │    │    ├── ma5800/
 │    │    ├── s-series/
 │    │    └── docs/
 │    ├── zte/
 │    ├── cisco/
 │    ├── datacom/
 │    ├── fiberhome/
 │    ├── vsol/
 │    ├── mikrotik/
 │    ├── sonicwall/
 │    └── hillstone/
 │
 ├── protocols/
 │    ├── bgp/
 │    ├── mpls/
 │    ├── ospf/
 │    ├── isis/
 │    ├── radius/
 │    ├── pppoe/
 │    ├── evpn/
 │    └── pbr/
 │
 ├── web-archive/
 │    ├── warc/
 │    └── extracted/
 │
 ├── forums/
 ├── pdf/
 ├── tor/
 ├── torrent-metadata/
 └── indexes/
      ├── opensearch/
      └── zoekt/
```

O detalhe mais importante é que as pastas seriam apenas armazenamento. A pesquisa de verdade viria dos índices e metadados.

A camada de inteligência que transforma isso em um pesquisador

O projeto que eu colocaria no topo dessa infraestrutura é **GPT Researcher** ou um agente semelhante construído com LangGraph. GPT Researcher é atualmente open source e foi projetado exatamente para receber uma pergunta, quebrá-la em subperguntas, pesquisar diversas fontes paralelamente, ler

documentos e produzir relatório com citações. Ele pode pesquisar web e documentos próprios e suporta retrievers como Google, Bing, DuckDuckGo e SearXNG, além de fontes conectadas por MCP. 〖cite turn205491search0 turn205491search1 turn205491search2〗

Existe também a implementação **Open Deep Research** da Hugging Face baseada em `smolagents`, criada como uma reprodução open source de sistemas de deep research; sua documentação descreve um agente que usa mecanismos de busca, navegador e execução de ferramentas para investigar uma pergunta. 〖cite turn205491search6〗 O antigo `open_deep_research` da LangChain também implementava pesquisa paralela com múltiplos provedores de modelos, APIs de busca e MCP, mas o repositório foi arquivado em agosto de 2026, então eu não o escolheria como base nova hoje. 〖cite turn205491search4 turn205491search5〗

A grande diferença é esta: **o agente não deve ser a sua base de dados**. Ele deve ser o investigador sentado em cima dela.

Quando você perguntar:

> “Encontre exemplos reais de redundância de dois concentradores PPPoE/BNG Huawei NE8000 autenticando no mesmo conjunto de RADIUS e usando BGP para failover; procure documentação, configurações, Git, fóruns, PDFs e versões arquivadas.”

o agente poderia automaticamente produzir consultas como:

```text
"NE8000" "radius-server group" BGP
"NE8000" PPPoE redundancy
"BRAS" Huawei "RADIUS" "BGP"
site:support.huawei.com NE8000 BNG redundancy
filetype:pdf NE8000 configuration guide BGP
"peer" "route-policy" "NE8000"
"radius-server shared-key" NE8000
```

e, simultaneamente, perguntar aos índices locais:

```text
Zoekt:
  "radius-server" AND "NE8000"

OpenSearch:
  semantic("redundância BNG PPPoE com dois roteadores e Radius")

Software Heritage:
  repositórios contendo strings de configuração Huawei

SearXNG:
  web pública em múltiplos motores

Common Crawl:
  páginas antigas que desapareceram do Google

Wayback:
  versões históricas de URLs conhecidas

Ahmia:
  somente se houver motivo legítimo para pesquisar fontes onion públicas

bitmagnet:
  somente metadata relacionada a datasets/documentação distribuída
```

O Wayback Machine também possui seu índice CDX, que permite localizar capturas de URLs por endereço, domínio, prefixo e data; isso é extremamente útil quando um tutorial antigo, fórum ou manual deixou de existir na web atual. 〖cite turn689589search8〗

O que eu construiria na prática

Se o objetivo é realmente criar sua própria “biblioteca técnica gigantesca”, eu faria **dois mecanismos

especializados compartilhando a mesma interface de pesquisa**, exatamente próximo do que você imaginou.

O primeiro seria o **Code Eye**: Software Heritage para cobertura histórica/global, ingestão seletiva de GitHub/GitLab/Gitea/Bitbucket, Zoekt como índice principal de strings/código e OpenGrok para navegação/cross-reference. Isso é tecnicamente capaz de trabalhar com milhões de repositórios, mas tentar reproduzir localmente toda a escala do Software Heritage significa entrar imediatamente na faixa de dezenas de terabytes apenas para estruturas de dados globais. 〖cite turn287925search0 turn287925search1〗

O segundo seria o **Web Eye**: SearXNG para descoberta imediata; Common Crawl como “oceano histórico”; Nutch/Heritrix para coleta própria; Crawl4AI para páginas modernas; Tika para documentos; Wayback para páginas mortas; Ahmia para universo onion publicamente descoberto; bitmagnet para metadata DHT; e OpenSearch como índice unificado. Cada componente resolve uma parte diferente que nenhum Google sozinho consegue cobrir. 〖cite turn689589search0 turn808118search9 turn721123search5 turn721123search8 turn205491search7 turn689589search4 turn490676search0 turn490676search4〗

Para armazenamento, eu não começaria tentando baixar os 10+ PB do Common Crawl nem os dezenas de terabytes do Software Heritage. Common Crawl permite consultar seu índice de URLs e baixar apenas os segmentos/registros relevantes; o Software Heritage também oferece APIs e datasets estruturados. Essa abordagem de “**descobrir globalmente, armazenar seletivamente**” é o que torna o projeto viável em um servidor de dezenas de terabytes em vez de exigir um datacenter. 〖cite turn808118search11 turn287925search1〗

O resultado pode, de fato, tornar-se algo muito mais poderoso para redes e telecom que uma busca convencional: um sistema em que você pergunta em português e ele pesquisa simultaneamente **código, PDFs, configurações, manuais, fóruns, páginas atuais, páginas apagadas, datasets públicos e fontes especializadas**, preserva o que encontrou, deduplica, classifica por fabricante/firmware/modelo e aprende progressivamente quais fontes são valiosas. A parte realista a corrigir na metáfora do “olho de Deus” é apenas a cobertura: nenhum sistema pode enxergar conteúdo privado, não publicado, protegido por autenticação, onion desconhecido ou algo que jamais foi capturado/indexado. Tor, em particular, foi deliberadamente projetado de maneira que conhecer todos os serviços não seja equivalente a enumerar a DNS pública. 〖cite turn942089search2 turn942089search4〗

A combinação que considero mais forte para o seu caso é, portanto, **Software Heritage + Zoekt** para o “Git inteiro”, e **SearXNG + Common Crawl + Nutch + Tika + OpenSearch**, complementados por **Wayback + Ahmia + bitmagnet**, para o “resto do mundo”. Sobre tudo isso, **GPT Researcher** funcionaria como o cérebro que decide onde pesquisar, cruza evidências e entrega a resposta com as fontes. 〖cite turn808118search3 turn808118search2 turn689589search0 turn808118search10 turn721123search5 turn689589search4 turn689589search6 turn490676search0 turn490676search5 turn205491search1〗